UNIVERSITI TEKNOLOGI MARA

ADAPTIVE THRESHOLD INTRUSION DETECTION USING RANDOM FOREST ON RASPBERRY PI FOR RESOURCE-CONSTRAINED NETWORKS

SYAKIR MUZAFFAR BIN SAIFUL BAHARI

THESIS SUBMITTED IN FULFILMENT OF THE REQUIREMENTS BACHELOR OF COMPUTER SCIENCE (HONS.) NETCENTRIC COMPUTING FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

JUNE 2026

SUPERVISOR APPROVAL

ADAPTIVE THRESHOLD INTRUSION DETECTION USING RANDOM FOREST ON RASPBERRY PI FOR RESOURCE-CONSTRAINED NETWORKS

By

SYAKIR MUZAFFAR BIN SAIFUL BAHARI

2025178227

This thesis was prepared under the supervision of the project supervisor, …… It was submitted to the Faculty of Computer and Mathematical Sciences and was accepted in partial fulfilment of the requirement for the degree of Bachelor of Computer Science (Hons.) Netcentric Computing.

Approved by

( )

Dr Siti Rahayu binti Abdul Aziz

Supervisor

26 June 2026

Table of Contents

CHAPTER 1 INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Scope 3

1.5 Significance 4

CHAPTER 2 LITERATURE REVIEW 5

2.1 Network Intrusion Detection System (IDS) 5

2.1.1 Definition and Purpose of IDS 5

2.1.2 Types of IDS 5

2.1.3 IDS Detection Approaches 5

2.1.4 Challenges in IDS Deployment 6

2.2 Cyberattacks in Home and Small Office Networks 7

2.2.1 Denial of Service (DoS) Attack 7

2.2.2 Port Scanning Attack 7

2.2.3 ICMP Flood Attack 8

2.3 Resource-Constrained Edge Devices 9

2.3.1 Definition and Characteristics 9

2.3.2 Raspberry Pi as an Edge Device 9

2.3.3 Challenges of Running IDS on Constrained Hardware 10

2.4 Machine Learning for Network Intrusion Detection 11

2.4.1 Overview of Machine Learning in Cybersecurity 11

2.4.2 Random Forest Algorithm 11

2.4.3 Feature Extraction and Selection for Network Traffic 12

2.4.4 Network Traffic Datasets for IDS Training 12

2.5 Threshold-Based Detection in IDS 13

2.5.1 Fixed Threshold Detection 13

2.5.2 Limitations of Fixed Threshold in Dynamic Networks 14

2.5.3 Adaptive Threshold Detection 14

2.5.4 Exponential Weighted Moving Average (EWMA) in Threshold Calculation 15

2.6 False Positive Problem in IDS 15

2.6.1 Definition and Impact 16

2.6.2 Alert Fatigue 16

2.6.3 Existing Approaches to Reduce False Positives 16
2.7 Related Works 17
2.7.1 Lightweight IDS in IoT Networks Using Raspberry Pi 3B+ 17
2.7.2 Energy Consumption of On-Device Machine Learning Models for IoT Intrusion Detection 18
2.7.3 Optimized Ensemble Deep Learning for Real-Time Intrusion Detection on Resource-Constrained Raspberry Pi Devices 19
2.7.4 Design and Implementation of a Machine Learning-Based Adaptive IDS on Raspberry Pi for Smart Home Network Security 19
2.7.5 A Cost-Effective Machine Learning Based Network Intrusion Detection System Using Raspberry Pi for Real-Time Analysis 20
2.7.6 Summary of Related Works 21
CHAPTER 3 METHODOLOGY 22
3.1 Introduction 22
3.2 Overview of Methodology 22
3.2.1 Waterfall Methodology 22
3.2.2 Project Workflow Diagram 23
3.3 Planning 25
3.3.1 System Architecture Overview 25
3.3.2 Dataset Selection (CIC-IDS2017) 25
3.3.3 Machine Learning Algorithm Selection (Random Forest) 27
3.3.4 Adaptive Threshold Mechanism Design (EMA) 29
3.3.5 Hardware Specifications (Raspberry Pi) 29
3.3.6 Software and Tools 30
3.3.7 Project Timeline 31
3.4 Design 32
3.4.1 System Overall Flowchart 32
3.4.2 System Architecture Diagram 33
3.4.3 Network Topology / Testbed Setup 34
3.4.4 Traffic Capture and Feature Extraction Design 35
3.4.5 Random Forest Model Design 37
3.4.6 Adaptive Threshold (EMA) Design 38
3.4.7 Database Design 40
3.4.8 IP Auto-Block (Mitigation) Design 40
3.4.9 Alert and Dashboard (Flask) Design 41
3.5 Development 44

3.5.1 Environment Setup 44
3.5.1.1 Raspberry Pi OS Configuration 44
3.5.1.2 Python Environment and Dependencies 44
3.5.1.3 SQLite Database Setup 45
3.5.2 Dataset Pre-processing and Feature Selection 45
3.5.3 Random Forest Model Training (Offline) 46
3.5.4 Model Deployment on Raspberry Pi 46
3.5.5 Live Traffic Capture and Feature Extraction 46
3.5.6 Adaptive Threshold Implementation 47
3.5.7 IP Auto-Block Implementation 48
3.5.8 Web Dashboard Development (Flask) 48
3.6 Evaluation and Testing 49
3.6.1 Testbed Configuration and Attack Simulation (hping3, nmap) 49
3.6.2 Detection Rate Experiment 50
3.6.3 False Positive Rate Experiment (Fixed vs. Adaptive Threshold) 50
3.6.4 CPU and Memory Utilization Experiment 50
3.6.5 System Latency Experiment 51
3.6.6 Performance Metrics and Evaluation Criteria 51
3.7 Summary 54
REFERENCES 55

LIST OF TABLES

Table 2.1: Raspberry Pi models variance in different articles 10
Table 2.2: Comparison between 4 commonly used datasets from reviewed literature 13
Table 2.3: Summary of Related Works 21
Table 3.1: Tasks and Deliverables of Each Phases 23
Table 3.2: Comparison between different datasets 26
Table 3.3: Comparison of Machine Learning Algorithms for IDS on Constrained Hardware 28
Table 3.4: Raspberry Pi Hardware Specifications 29
Table 3.5: Software and Tools Used 30
Table 3.6: Gantt Chart of Project Timeline 31
Table 3.7: Features Extracted from Traffic 35
Table 3.8: Core Python Dependencies Used 44
Table 3.9: Tools and Commands for Attacks Used 49
Table 3.10: Observed Performance and Accuracy Results 52

LIST OF FIGURES

Figure 3.1: Overall System Flowchart 33
Figure 3.2: System Architecture Diagram 34
Figure 3.3: Network Topology/Testbed Setup 35
Figure 3.4: Random Forest Training Pipeline 37
Figure 3.5: EMA Adaptive Threshold Flow Diagram 39
Figure 3.6: Database Design 40
Figure 3.7: User Interface of Live Alert Page 42
Figure 3.8: User Interface of Traffic Log View Page 42
Figure 3.9: User Interface of System Status View Page 43
Figure 3.10: User Interface of IP Auto-Block Panel Page 43

CHAPTER 1

INTRODUCTION

1.1 Background

Research on cybersecurity of home and small office (SOHO) networks has become a core topic in the current cybersecurity field. Musthafa et al., 2025 point out that the popularization of smart homes, the explosive growth of IoT devices, and the normalization of the global remote work model have drastically elevated the attack risk of these small networks that lack professional protection. Common threats include Denial-of-Service (DoS) attacks, port scanning, network flooding, and others.

Among mainstream defense tools, the Intrusion Detection System (IDS) is the most widely applied. However, the 2020 study by Devan & Khare clearly states that all existing mainstream IDS are designed for large enterprises that own dedicated hardware, full-time IT teams, and sufficient budgets, with high deployment costs and strict operational hardware requirements, making them completely incompatible with the various low-power, low-cost terminal devices used in home networks.

To solve the IDS compatibility challenge, the mainstream technical option in current academia is the machine learning-based Random Forest algorithm: compared with single decision trees, it has higher accuracy and is less likely to suffer from overfitting; compared with deep learning models, it has lower demands for hardware resources and is easier to deploy on small devices. When paired with the Synthetic Minority Oversampling Technique (SMOTE), it can also effectively address the widespread dataset imbalance problem in real-world networks.

Academia has also conducted multiple optimization studies around this direction in the past: Liu et al., 2021 proposed combining genetic algorithms to identify various attack variants; Disha & Waheed, 2022 further improved the algorithm's running speed by reducing the number of model input features; and tests by Zhang et al., 2024 also verified the initial usability of these optimization schemes in SOHO network scenarios.

However, as a typical representative of home edge devices, a single Raspberry Pi costs less than RM500. The 2025 study by Wijethilaka et al. points out that such devices face strict resource constraints, and the lightweight IDS currently developed to fit these devices generally have core flaws: tests by Le et al., 2022 found that the fixed traffic thresholds adopted by these systems often misclassify normal traffic peaks generated by scenarios such as device backups, system software updates, and high-definition video calls as attack traffic. Since home networks have no full-time staff to investigate and resolve false alarms, this directly leads to a sharp decline in the reliability of the entire defense system.

The latest 2025 study by Adrian et al. also confirms that the high false positive rate caused by fixed thresholds is the core problem that urgently needs to be addressed for the implementation of lightweight IDS in home scenarios. This project adopts the Raspberry Pi as its hardware carrier, builds a lightweight intrusion detection system equipped with the random forest algorithm, and replaces fixed thresholds with dynamic thresholds, with the goal of reducing the false alarm rate while ensuring the system's ability to identify real attacks.

1.2 Problem Statement

Although it is well recognized that machine learning could be a solution to develop lightweight IDS for resource-constrained devices such as the Raspberry Pi, many existing solutions have significant drawbacks for deployment in real-world scenarios. The first thing is that most lightweight IDS applications have a fixed threshold for classification, which is used to determine whether to trigger an alert or not (Gupta et al., 2021). This means that the network is equally vulnerable during the time of a traffic spike, for instance when customers are installing new software or backing up their data to the cloud or when a customer is having a video call, even when the network is busy. (Adrian et al., 2025; Wijethilaka et al., 2025)

Secondly, this fixed threshold also leaves a detection gap during quiet hours like late at night when a threshold set for more active time may be too high to detect slow or subtle attacks, resulting in attacks going undetected (Gupta et al., 2021). The static decision boundaries are a general shortcoming and have been shown to be poor if the data pattern changes over time (Le et al., 2022).

Moreover, while there are some works that address the size of the model and the inference time to make IDS more realistic on limited devices (Tekin et al., 2023), threshold adaptability for network changes has been relatively under-investigated. In the implementations reviewed for this proposal, no one uses a threshold that changes based on the current network behaviour, but all use a threshold that is fixed for the duration of the calibration phase and remains constant thereafter.

Overall these constraints indicate the need for a lightweight IDS that can be deployed on the types of hardware available, but is sensitive to network changes and does not result in too many false alerts during periods of heavy traffic and yet remains effective during periods of low traffic. Each of these gaps are solved with the proposed project which consists of an IDS running on the Raspberry Pi using Random Forest and an adaptive threshold mechanism based on Exponential Moving Average (EMA) that continuously adjusts the detection boundary according to the current network behaviour.

1.3 Objectives

To design a lightweight ML-based IDS architecture with an adaptive threshold mechanism for deployment on Raspberry Pi as a network gateway.

To develop a working IDS system on Raspberry Pi that captures live network traffic, extracts features, classifies traffic using a Random Forest model, and adjusts the detection threshold dynamically based on a rolling network baseline.

To evaluate the system by comparing the false positive rate and attack detection rate of the adaptive threshold approach against a fixed threshold approach under simulated normal and attack traffic conditions.

To implement an optional IP auto-blocking mitigation mechanism that temporarily restricts network access for source IP addresses flagged as attacks, controllable by the user through the system dashboard.

1.4 Scope

Target Users:

End-users / Home Network Users: The primary users of this system are individuals who want basic network protection for their home or small office network. This includes homeowners with smart devices connected to their network, remote workers, and small business owners who need affordable network security without relying on expensive enterprise solutions.

IT Administrators / Cybersecurity Researchers: The secondary users are technically inclined individuals such as network administrators managing small office environments and cybersecurity students or researchers who want to study lightweight IDS behaviour on constrained hardware.

Software and Tools:

Front-end: Web-based dashboard (Flask), shows real-time alerts, traffic logs, and system status through a browser on the local network.

Database: SQLite, stores alert logs and traffic records locally on the Raspberry Pi.

Back-end: Python, handles traffic capture, feature extraction, machine learning, and threshold calculation.

Hardware Platform: Raspberry Pi 4 Model B, used as the network gateway device that runs the IDS, dashboard, and database.

Algorithm and Technology Used:

Machine Learning Algorithm: Random Forest Classifier, used to decide whether network traffic is normal or a potential attack.

Adaptive Threshold Mechanism: Exponential Moving Average (EMA), used to adjust detection sensitivity automatically based on current traffic conditions, reducing unnecessary false alarms.

Attack Simulation: hping3 is a tool to send a rapid, continuous stream of TCP packets or ICMP ping packets without any of the connection steps that would normally be completed, simulating a Denial of Service attack or Ping Flood. nmap is a port scanner used to quickly scan a range of ports on the target device without completing a normal connection, simulating a port scan an attacker might use to locate open services. Both tools are executed from a separate attacker laptop on the test network, passing traffic through the Raspberry Pi gateway for the IDS to capture and classify during tests.

Dataset:

The Canadian Institute for Cybersecurity has developed a publicly available network traffic dataset (CIC-IDS2017) for training and testing the Random Forest classifier offline prior to deployment. The dataset consists of labeled flow records of a variety of attacks, including DoS, DDoS, Port Scanning, Brute Force Attacks, as well as realistic background traffic. It is selected because the features used in training it were extracted using the same tool (CICFlowMeter) that is used to extract the features during live deployment on the Raspberry Pi, thus ensuring consistency of the training data and live traffic.

Target Location:

A simulated home or small office network environment set up in a lab. The Raspberry Pi acts as a gateway device placed between a test laptop and a router, so that all network traffic passes through it for monitoring.

1.5 Significance

To the cybersecurity field: This project contributes evidence that adaptive threshold detection is computationally feasible on resource-constrained edge hardware. All existing related work reviewed uses fixed thresholds. This project fills that documented gap and provides measurable comparison data between fixed and adaptive approaches.

To home and small business users: Most small networks today have no IDS protection at all because enterprise solutions are expensive and complicated to manage. This project demonstrates a low-cost alternative that can run on a RM500 device and provide meaningful protection against common attacks like DoS and port scanning.

To future researchers: The results of this project, specifically the false positive rate comparison between fixed and adaptive thresholds under different traffic conditions, provide a baseline that future researchers can build on. The system architecture is modular, meaning future work can extend it with more attack types, better models, or multi-device configurations.

CHAPTER 2

LITERATURE REVIEW

2.1 Network Intrusion Detection System (IDS)

The rest of the literature review will draw on this section to give the background information in IDS. It explains the concepts of an IDS and its necessity, the various types of IDS and deployment patterns, two major approaches to detection in practice, and the major problems that make IDS deployment difficult, especially in resource-constrained environments.

2.1.1 Definition and Purpose of IDS

A network intrusion detection system (IDS) is a type of security mechanism that monitors the network traffic and detects suspicious malicious intrusion activities. According to the study undertaken by Thakkar & Lohiya (2020), while firewalls are designed around the access control concept and restrict access by network boundaries, IDS is designed around the principle of monitoring traffic continuously and alerting whenever anomalies are detected. It aims to provide live feeds and visibility of the activities in the network, identifying threats at an early stage and preventing significant losses.

Automated IDS is the backbone of defense for home networks and small office networks without a dedicated security team. In addition to the observed rising popularity trend of IoT and edge computing endpoints by Kuthada et al. (2021), lightweight IDS solutions are becoming increasingly popular off the traditional enterprise architecture.

2.1.2 Types of IDS

IDS can be divided into two types of usage: hosestat and nettat.Intrusion Detection Systems (IDS) are divided into two types of usage: hosestat and nettat. According to Wijethilaka et al., a study from 2025, Network Intrusion Detection Systems (NIDS) are installed at strategic points, like gateways and routers, that can monitor traffic in all network segments and detect attacks such as port scanning and denial of service flooding. Host Intrusion Detection Systems (HIDS) only run on individual endpoints and provide the ability to see what's happening on the host itself in great detail, but can't see into the rest of the network.

The NIDS architecture used in this project is an architecture where the gateway on which all the network traffic flows is a Raspberry Pi, where no terminal proxies need to be installed, since the entire flow is visible.

2.1.3 IDS Detection Approaches

IDS systems use two main methods of detection. Signature-based detection compares the traffic it observes to a database of signatures of known attacks. Although it is precise for known threats,

it cannot detect new attack variants, and signature updates are still necessary, which must be done manually (Thakkar & Lohiya, 2020).

Anomaly-based detection creates a baseline of normal network behaviour and alerts when the traffic seen is significantly different from the baseline. This method has the ability to detect unknown attacks, but in the past, it resulted in higher false positive rates, especially when normal traffic in the system is irregular (Halimaa & Sundarakantham, 2019). The most prominent technique for anomaly-based IDS is ML, which is used to train classifiers on labeled traffic data to learn decision boundaries with high detection accuracy, significantly outperforming the accuracy of rule-based IDS (Musthafa et al., 2025).

This project uses an Adaptive threshold mechanism specifically to tackle the false positive issue in the context of anomaly detection and applies a machine learning approach based on the Anomaly-based approach and the use of a Random Forest Classifier.

2.1.4 Challenges in IDS Deployment

There are a number of common issues involved in implementing IDS in practice. The enterprise-grade solutions provide high-performance detection capabilities but involve substantial hardware costs and specialized IT skills which are not available to home users and SMBs (Kuthada et al., 2021). This has led to an increasing number of studies on IDS implementations on limited hardware like single board computers.

One of the most significant ongoing issues of operation is the false positive: a false alarm alert is triggered, leading to alert fatigue and loss of operator trust (Adrian et al., 2025). On the other hand, an over-sensitive system can trigger a false alarm to minor or low rate attacks. The common trend in the literature is that all of the lightweight IDSs evaluated are based on fixed classification thresholds, which are unable to adjust to the natural variation of legitimate traffic volume as observed in the literature (Wijethilaka et al., 2025; Musthafa et al., 2025; Kuthada et al., 2021). This structural constraint is directly used as a driving force for the adaptive threshold mechanism proposed in this project.

2.2 Cyberattacks in Home and Small Office Networks

In this section, we will look into the three types of attacks this project will focus on: Denial of Service, port scanning, and ICMP flood. For each attack, it will be explained how it functions, what its effect is on the attacked system and what traffic patterns it generates that allow it to be detected by a flow based IDS. It is essential to have knowledge of these attack signatures for justifying the features that have been selected for the Random Forest classifier in Chapter 3.

2.2.1 Denial of Service (DoS) Attack

Denial-of-Service (DoS) attacks are a common attack vector in the realm of cybersecurity, the primary aim of which is to disrupt the availability of a target system. This provides a clear fundamental distinction between DoS attacks and intrusion attacks, where the fundamental requirement is to gain access to something without permission. A research published in 2021, by the Kuthada team, clearly explained the core differences between these two types of attacks.

Distributed Denial-of-Service (DDoS) attacks, as one of the main types of DoS attacks, has been gaining popularity in the industrial world because of its multi-source coordinated attack. Ali et al., however, found in their 2023 study that the single-source DoS attack was still a prevalent, high-frequency, and low-complexity attack in home and small office environments.

The most popular DoS attack type studied in academic research are currently TCP SYN Flood attacks. This attack uses a large number of SYN packets, fake source IP addresses to deplete resources of the target's half-open connection queue, which requires a three-way handshake to maintain a reliable TCP connection. The characteristics of its traffic can be summarized as: a sudden surge in the number of packets, a high percentage of SYN packets to established connection, and short flow durations.

Given the characteristics of the attacks discussed above, this project aims at detecting attacks using the concept of a flow-level feature analysis. It takes the public CIC-IDS2017 dataset to train a model and a random forest classifier to implement the identification of attacks.

2.2.2 Port Scanning Attack

Port scanning is a reconnaissance technique that involves a series of probes to an Internet host to determine what services are open and vulnerable. Although port scanning itself is not an attack that will make immediate harm, it is believed to be an initial attack and an important step in the kill chain, which is the process used by an attacker to map out the network prior to a targeted exploit (Abu Bakar & Kijsirikul, 2023). Successful reconnaissance can follow with exploits like brute-force logins, service exploitation (unpatched) and lateral movement in home or small office environments where devices like NAS drives, cameras, and smart home controllers provide a network service.

Port scans generate a unique traffic signature which varies significantly from normal communication activity. A scanning host usually scans a large number of destination ports in a short period of time, has very low bytes or duration of flows, and a high ratio of TCP RST or ICMP unreachable flows to established flows (Wu et al., 2023). The flow-level characteristics of this behaviour, such as destination port entropy, flow duration or the ratio of bytes to packets, are ideal for detection by machine learning classifiers.

In this project the nmap tool is used to simulate controlled attacks; it is the de facto standard in penetration testing practice and academic IDS evaluation. It can generate various types of scans, such as TCP SYN scans, TCP connect scans, and UDP scans, generating slightly different traffic signatures, thus offering a realistic and varied test case for assessing the performance of port scan detection systems (Abu Bakar & Kijsirikul, 2023).

2.2.3 ICMP Flood Attack

ICMP flood (Ping flood) is one of the common volume-based DoS attacks. This attack consumes network and system resources by sending huge volumes of ICMP Echo Request packets towards the target and a study conducted in 2021 by Kuthada et al. proved that this attack will eventually make all external services of the target unavailable.

The signature of this attack is clearly identifiable in terms of the flow level characteristics: a high rate of packet sending from one source, all packets have a uniform constant size which corresponds to the fixed size of the ICMP header and payload, and the length of time the attack is sustained is different from the normal interval of the diagnostic ping traffic.

In this study, we establish a 5 second frame of time to consider as our flow window, use data available in public data set CIC-IDS2017, and apply random forest classifier to detect this attack effectively.

2.3 Resource-Constrained Edge Devices

This section discusses the hardware context in which this project operates. It clarifies the notion of resource constrained edge devices, introduces the Raspberry Pi as the representative platform of this class of devices, and discusses the challenges of implementing an IDS on such devices. The constraints explicitly stated in this chapter directly influence the design decisions taken in chapter 3.

2.3.1 Definition and Characteristics

Resource-constrained edge devices are computing systems that, by design, operate under significant limitations in processing power, memory, storage, and energy capacity (Canavese et al., 2024). Unlike general-purpose computers or cloud servers, these devices are built for cost-effective, localised deployment at or near the point of data generation — the network edge — rather than relying on centralised infrastructure for computation. Common examples include IoT sensors, smart home gateways, budget routers, and single-board computers such as the Raspberry Pi.

The defining characteristics of such devices are their restricted hardware profiles: limited CPU clock speed, small RAM footprints typically measured in gigabytes or less, constrained storage, and in many cases a dependence on low-power consumption to enable battery-operated or always-on operation. These limitations stand in direct contrast to the computational demands of modern security software. As IoT adoption has accelerated — with an estimated 14.4 billion active endpoints recorded in 2022 and projections approaching 30 billion connected devices by 2027 — the security implications of deploying constrained devices at scale have become a central research concern (Canavese et al., 2024).

2.3.2 Raspberry Pi as an Edge Device

The Raspberry Pi is a single-board computer (SBC) originally developed to promote accessible computing education. Its combination of low cost, ARM-based processing, and a mature open-source software ecosystem has made it widely adopted as a representative platform for edge computing research and IoT gateway prototyping. Tekin et al. (2023) specifically categorise the Raspberry Pi among IoT prototyping devices capable of running machine learning models — a capability absent from many commercial off-the-shelf constrained devices.

Within the IDS research literature, the Raspberry Pi is consistently used not as a consumer product in its own right but as a validated proxy for the broader class of constrained ARM-based edge devices, including budget routers, OpenWrt appliances, and embedded IoT gateways, that share comparable hardware profiles (Wijethilaka et al., 2025; Musthafa et al., 2025). Findings regarding model performance, inference latency, and resource utilisation on Pi hardware therefore transfer meaningfully to these real-world deployment targets.

Multiple models of the Raspberry Pi have been evaluated in IDS contexts across the five reviewed papers. The Table 2.1 below reflects only what is explicitly documented in those publications:

Table 2.1: Raspberry Pi models variance in different articles

MODEL	CPU SPECIFICATION	RAM (GB)	CITATION
Raspberry Pi 3B+	1.4 GHz ARM Cortex-A53 (quad-core)	1	Kuthada et al. (2021); Musthafa et al. (2025)
Raspberry Pi 4B	1.5 GHz ARM Cortex-A72 (quad-core)	8	Tekin et al. (2023)
Raspberry Pi 4B	1.5 GHz ARM Cortex-A72 (quad-core)	2/4/8	Wijethilaka et al. (2025)

2.3.3 Challenges of Running IDS on Constrained Hardware

When deploying real-time Intrusion Detection Systems (IDS) on Pi-grade edge hardware such as the Raspberry Pi, existing academic literature has collectively verified three core constraining challenges, and the technology selection logic of this study is derived directly from these challenges.

The first challenge is the trade-off between inference latency and CPU power consumption. Tekin et al. (2023) found that on Pi hardware, the inference time and power consumption of random forests are far higher than those of decision trees, and training random forests directly on IoT device ends is not energy-efficient. This contradiction between accuracy and operational overhead only exists on Pi-grade hardware, and does not appear on server-grade hardware.

The second challenge is model size and memory limits. Empirical data from Musthafa et al. (2025) shows that the Raspberry Pi 3B+, which is only equipped with 1GB of RAM, cannot accommodate an unoptimized LSTM model; the model must be compressed from 2581KB to 235KB via pruning and quantization, a 90.9% size reduction, to be deployable.

The third challenge is limits on sustained workload intensity. Musthafa et al. (2025) observed that when Pi hardware runs an IDS continuously, its CPU usage peaks at 111.8% and rises gradually as workload intensity increases; Tekin et al. (2023) add that long-term operation of machine learning inference on Pi devices leads to measurable energy losses, and always-online deployment requires refined resource management.

Based on the above constraints, this study defines two core design decisions. First, we select random forests rather than deep ensemble models or neural networks. The supporting basis for this choice is the outstanding on-Pi accuracy of random forests verified by Wijethilaka et al. (2025). Second, we eliminate the real-time retraining function, in line with the conclusion proposed by Musthafa et al. (2025) that online model updates incur excessively high resource overhead and are unsuitable for this type of hardware. All technology selections in this work strictly conform to the previously verified deployment constraints.

2.4 Machine Learning for Network Intrusion Detection

This section surveys the use of machine learning techniques for the intrusion detection problem. It explains the concept of ML in Cyber Security, its applicability to constrained hardware deployment, how flow based features are extracted from network traffic and the datasets frequently used for the training and evaluation of ML-based IDS. These topics, taken together, warrant the algorithm and dataset selections in this project.

2.4.1 Overview of Machine Learning in Cybersecurity

The main reason for the rise of machine learning as the dominant paradigm for anomaly-based intrusion detection is the ability of machine learning to learn discriminative patterns from labelled network traffic data without manually writing rules. Among the supervised learning methods, the most frequently used are the ones employed in the current IDS research and yield high accuracy for well-known attack patterns (Kumar et al., 2023). The overall process of the general pipeline can be said to be of four stages: (1) Packet Capturing, (2) Traffic Analysis and Filtering, (3) Pattern Matching with trained model, (4) Alert Generation (Thakkar & Lohiya, 2020).

One of the major advantages of the ML approach is the offline-train, online-infer paradigm. Training needs a huge amount of computation power and data storage, which is not suitable for edge devices, and is usually done on powerful computing resources like cloud services or local machines (Tekin et al., 2023). The trained model is then deployed to the constrained device for real time inference. Musthafa et al. (2025) show this explicitly, they prune and quantise their stacked LSTM training model before deploying it on their Raspberry Pi 3B+ for inference.

The IDS that is based on ML is more robust against modified attacks than signature based IDS. The signature of an attack can be completely altered by using a minor change to the attack tool making signature-based detection ineffective (Pinto et al., 2023). An ML-based classifier on the other hand, will use statistical features like packet rates, TCP flag distributions, byte counts and flow duration, which will remain anomalous even when the attack tooling changes (Wijethilaka et al., 2025). This property is crucial for the motivation of using ML in this project.

2.4.2 Random Forest Algorithm

Random Forest is a typical ensemble learning algorithm. Its core mechanism is to construct multiple independent decision trees during the training stage: each tree is generated from a random bootstrap sample of the training dataset, each split node only selects features from a random feature subset, and the final classification output is determined by majority voting across the entire forest. A 2025 study by Wijethilaka et al. confirmed that this bagging strategy reduces model variance, improves stability, and prevents overfitting compared to a single decision tree.

On the NF-UQ-NIDS dataset, the team used Principal Component Analysis (PCA) and Recursive Feature Elimination (RFE) to screen core attack features including TCP flags, flow

duration, number of inbound packets, and number of outbound packets, and verified the algorithm's generalization ability. A 2023 study by Tekin et al. further validated its deployment advantages in hardware-constrained edge intrusion detection systems (IDS): unlike neural networks, which require extensive gradient computation for inference, tree-based algorithms require no multiplication operations during inference, with low and predictable computational costs.

The team's measurements on Raspberry Pi showed that Random Forest had a training time of 624.43 seconds, while a single decision tree took only 40.86 seconds; both achieved 99% accuracy, ranking in the top accuracy tier alongside ANN and k-NN. The characteristic that edge scenarios only need to run inference while model training can be completed offline makes this trade-off in training duration completely reasonable.

This project adopts the aforementioned core feature categories, uses Random Forest's native Gini importance score to validate the 12 screened features, which can fully support the real-time classification requirements of edge IDS.

2.4.3 Feature Extraction and Selection for Network Traffic

Flow-based feature extraction aggregates raw packet captures into statistical summaries describing the behaviour of a network connection over a defined time window. Rather than classifying individual packets, which is computationally prohibitive on constrained hardware, IDS systems group packets into flows and compute statistical features over those groups. Kuthada et al. (2021) highlight features such as transaction bytes and packet arrival rates as among the most discriminative for lightweight IDS on Raspberry Pi hardware.

Effective IDS features capture both volume characteristics such as packet counts and byte totals, and behavioural characteristics such as inter-arrival times, flag distributions, and flow duration. For their Pi-based NIDS, Wijethilaka et al. (2025) processed live packet streams using pyshark and tcpdump, applying PCA and RFE on the NF-UQ-NIDS dataset to identify critical features including TCP flags, flow duration, and incoming and outgoing packet counts. Tekin et al. (2023) further confirm that computational complexity scales with the number of input features, and that reducing the feature set is essential for minimising power consumption and inference latency on IoT devices.

This project extracts 12 features from 5-second flow windows aggregated per source IP, aligned with the feature categories present in CIC-IDS2017, to ensure consistency between the offline training distribution and the live deployment environment.

2.4.4 Network Traffic Datasets for IDS Training

The choice of training dataset significantly affects the performance and generalisability of an ML-based IDS. A well-constructed dataset must include realistic benign traffic, a diverse range of labelled attack types, and features extractable using tools available during live deployment.

The Table 2.2 below compares four commonly used datasets as documented across the reviewed literature:

Table 2.2: Comparison between 4 commonly used datasets from reviewed literature

DATASET	YEAR	ATTACKS INCLUDED	FEATURE EXTRACTION TOOL	CITATION
CIC-IDS2017	2017	DoS, DDoS, Brute Force, Port Scan, Botnet, Web Attacks	CICFlowMeter	Thakkar & Lohiya (2020)
UNSW-NB15	2015	DoS, Reconnaissance, Fuzzers, Exploits	Argus, Bro-IDS	Kuthada et al. (2021); Kumar et al. (2023)
NSL-KDD	2009	DoS, Probe, R2L, U2R	Pre-extracted	Thakkar & Lohiya (2020)
KDD99	1999	DoS, Probe, R2L, U2R	Pre-extracted	Thakkar & Lohiya (2020)

This project uses CIC-IDS2017 because it was generated and labelled using CICFlowMeter, producing 80 flow-level features from realistic simulated traffic (Thakkar & Lohiya, 2020). CICFlowMeter can equally be applied to live packet captures, meaning the feature extraction process during deployment mirrors the process used to construct the training data. This training-deployment consistency is a deliberate design choice to reduce distributional mismatch between the training environment and real-world inference on the Pi. The 12 features used in this project are a selected subset of the broader 80-feature space, chosen to balance detection coverage with the inference latency constraints documented by Tekin et al. (2023).

2.5 Threshold-Based Detection in IDS

This section examines how thresholds are used to make the final detection decision in an ML-based IDS. It covers how fixed thresholds work and why they are the current standard in lightweight IDS implementations, the limitations of fixed thresholds in dynamic network environments, how adaptive thresholds address those limitations, and how the Exponential Weighted Moving Average technique is used specifically to compute a dynamic threshold. This section directly establishes the research gap that this project addresses.

2.5.1 Fixed Threshold Detection

The classification threshold is the decision boundary for the output score of the model in ML-based IDS, which is used to classify a flow as an attack or as normal traffic. The fixed threshold approach is a method where this threshold is established during the initial development or calibration of the model and is then fixed for the duration of deployment, irrespective of network conditions changes (Wijethilaka et al., 2025; Kuthada et al., 2021).

Fixed thresholds are easy to implement and do not need to be computed at runtime other than the classifier itself. This is the trend that is evident in the literature reviewed. The main implementation papers are dedicated to addressing different hardware challenges: model compression via pruning and quantisation (Musthafa et al., 2025); energy efficiency benchmarking (Tekin et al., 2023); and feature selection to lower dimensionality for Pi hardware (Kuthada et al., 2021; Wijethilaka et al., 2025). In both cases, a fixed classification

threshold is used at the decision layer without changes. This is a confirmation of what is already the standard practice for IDS implementations of the Pi class.

2.5.2 Limitations of Fixed Threshold in Dynamic Networks

One of the drawbacks of fixed thresholds is that they cannot adjust to the natural variation of network traffic. The traditional threshold selection methods are mainly based on empirical or experimental results, which is hard to solve the temporal and dynamic characteristics of network traffic (Yu et al., 2024). Network activity is not constant and will vary throughout the day, week and with certain events like software updates, backups, etc. If the threshold is set under one set of conditions, it will not be calibrated properly if the conditions change.

Classifier scores for normal traffic can also increase towards the detection threshold when there is an unusually high volume of legitimate traffic. If traffic levels are low, subtle low-rate attacks can create scores that are below a threshold set for busier traffic and cause false negatives. Albasheer et al. (2022) report that in practice, up to 99% of all alerts generated by NIDS were false positive, which is a major issue for operators to lose trust in the tool and suffer from alert fatigue.

The closest implementation to dynamic conditions is that of Musthafa et al. (2025), who have set up an Apache Kafka-based testbed for simulating variable traffic loads, delays and multiple simultaneous attack sources. But their system still uses a fixed classification threshold for all these tests. The threshold does not vary with the volume of traffic. This is crucial: simulating dynamic traffic as an evaluation condition is not the same as implementing a mechanism that adjusts the detection boundary to simulate those conditions. The adaptive threshold mechanism proposed in this project is directly driven by this gap that has been observed in all the implementations reviewed.

2.5.3 Adaptive Threshold Detection

With adaptive threshold detection – instead of using a fixed threshold, the decision boundary is automatically adapted to the network conditions observed. Instead of using a fixed threshold value, an adaptive system keeps a running average of the current traffic level, and calculates the threshold based on that running average. The threshold increases when traffic is heavy, but legitimate, which decreases the number of false positives, and decreases when traffic is light, which increases the number of false positives.

The main benefit over fixed thresholding is that it eliminates the need to account for the conditions at calibration time. Fixed threshold methods need to be retuned manually if the network behaviour changes significantly, which is impractical in an always-on deployment without dedicated administrators. This is supported by Pinto et al. (2023), who have said that using expert labelling and manual rules is becoming increasingly impractical as attackers continue to evolve their techniques and traffic grows at an ever-increasing rate.

It should also be noted that the adaptive threshold mechanism used in this project is different from the use of adaptive by Adrian et al. (2025). They work by continually learning from new

data, updating their detection models through retraining, hence their system is described as adaptive. This is another thing altogether. Retraining involves updating the classifier, while adaptive thresholding involves adjusting the decision boundary to be applied to the output of a fixed classifier. The novelty of this project is in the latter, which is not implemented in the reviewed papers..

2.5.4 Exponential Weighted Moving Average (EWMA) in Threshold Calculation

Exponentially Weighted Moving Average (EWMA) is a time series smoothing method that calculates a weighted average of past observations, with weights decreasing exponentially as time goes back. This property is suitable for use in EWMA, as it is an ongoing and changing baseline. It reacts to real traffic changes and dampens the impact of short-term spikes that would otherwise cause an arithmetic average to be misrepresented.

Yu et al. (2024) proposed an enhanced EWMA model that works in fixed time windows to determine dynamic thresholds for anomaly detection in the context of network anomaly detection. Their algorithm calculates a fluctuation range F, which is a multiple of the standard deviation of the traffic scores over a short time, and thus forms a normal operating interval around the current baseline. They found that this is a better way to adapt to the real-world time-varying and time-sensitive nature of network traffic than fixed threshold-based methods.This project applies the same principle at the IDS decision layer, computing the adaptive threshold as:

threshold = baseline_mean + (k x baseline_std)

where the baseline mean and standard deviation are updated using EWMA after each observation window. The coefficient k serves the same functional role as the multiplier in Yu et al. (2024). The smoothing coefficient beta controls the responsiveness of the baseline to recent traffic changes. A lower beta produces a more stable but slower-reacting threshold, while a higher beta tracks recent behaviour more closely at the cost of greater sensitivity to short-term fluctuations. This formulation allows the IDS to remain calibrated to current network conditions without human intervention, directly addressing the limitation identified in Section 2.5.2.

2.6 False Positive Problem in IDS

This section covers the false positive issue, the main motivation which is the adaptive threshold mechanism proposed in this project. It describes what false positives are and why they are important, discusses alert fatigue and the impact on the usefulness of an IDS and what approaches the current research has taken to mitigate false positives. This section reveals that although feature selection and model-level methods are well explored, threshold-level are relatively underexplored.

2.6.1 Definition and Impact

In the context of IDS, a false positive is when the system alerts on traffic that is not malicious. The classifier has rated a normal flow as being a threat and above the detection threshold, so it is considered an attack. A false negative occurs when a legitimate attack flow fails to meet the threshold, and is not detected.

In practice, error of the false positive type is more prevalent and problematic than error of the false negative type, although both types have operational consequences. As reported by Albasheer et al. (2022), as many as 99% of alerts generated by NIDS deployments are false alarms – for every real threat alert, a lot of others will need to be investigated only to be ruled out as false. Tariq et al. (2025) also estimate the burden of this false positive, stating that analysts spend more than one-quarter of their working time specifically dealing with false positive alerts, and Yang et al. (2022) confirm that the overwhelming number of false positive alerts consumes a tremendous amount of resources in security operations.

The repercussions are far-reaching. In some situations, legitimate users or services can be denied access as a result of false positives and automated blocking actions like IP banning with iptables. This type of collateral disruption can cause the usefulness of the IDS to be lost for a long time in a home or small business setting where there is no IT personnel to monitor alerts.

2.6.2 Alert Fatigue

Alert fatigue is when security personnel become desensitized to alerts because they receive so many of them, especially when most of them are false positives. Tariq et al. (2025) cite excessive alerts as one of the primary reasons for desensitisation, burnout, and frustration among security analysts, with a 2023 IBM SOC survey showing that 76% of SOC teams reported that low priority and false positive alerts delayed their threat response. Similarly, Yang et al. (2022) report that alert burnout and eventual desensitisation result from massive volumes of false alerts and will directly result in missing or delayed true alerts.

These are enterprise class SOCs, but the same principle holds true and is even more relevant in resource-constrained environments like home networks and small offices. The environments are usually without a dedicated individual to review alerts. A false positive IDS in this environment will probably be ignored, disabled, or not configured at all. This disparity between enterprise level security tooling and the requirements of smaller user communities is cited as a driver for lightweight, reliable IDS research by both Kumar et al. (2023) and Adrian et al. (2025). The consequence of this alert fatigue in these situations is that an inadequately configured IDS system offers little or no security value when it's in place.

2.6.3 Existing Approaches to Reduce False Positives

Several approaches have been proposed in the literature to address the false positive problem in IDS. The most frequently used ones are feature selection, optimisation of the model, post-processing filters, and thresholding.

By using feature selection, the false positive rate is decreased as the classifier is only trained on the traffic attributes that are most discriminative, and not on noisy traffic attributes that would

lead the classifier to misclassify benign traffic. Kuthada et al. (2021) use CFS to remove all the irrelevant and redundant features, resulting in a light-weight model without being sensitive to uninformative traffic characteristics. Similarly, Wijethilaka et al. (2025) employ PCA and RFE to determine the feature subset that best predicts the attack behaviour, obtaining a much lower false positive rate as a consequence.

Optimisation at the model level focuses on the false positives by making the classifier more precise. Increased class distribution balance in the training data by using SMOTE helped in reducing false positive rate, as it made the classifier less biased towards the majority normal class, according to the study by Wijethilaka et al. (2025). In addition, Pinto et al. (2023) point out that ensemble techniques have been shown to achieve consistently high precision and F1 score over all metrics, and that supervised classifiers can achieve high accuracy and low false positive rates for well-known attack types when trained correctly.

Post-processing filters are applied to the output of the classifier and not to the model itself. According to Albasheer et al. (2022), alert correlation is a specific type of IDS post-processing that is used to verify and/or filter alerts before they are presented to the operator, to eliminate duplicate and low-confidence alerts from reaching the analyst.

A lesser known but growing method is threshold adjustment. Yu et al. (2024) classify direct threshold setting as an independent architectural component from feature extraction and model training, and suggest that static thresholds are structural causes of false positives in dynamic traffic settings. They suggest a method called dynamic thresholding as a decision level approach to minimizing misclassification, which is independent of the decision function's internal parameters.

This project is focused on this decision-layer approach to address the false positive issue. The adaptive EWMA-based threshold mechanism automatically adjusts the detection boundary to suit the traffic situation, without changing the classifier or the features themselves. It is a complementary and different contribution to the feature selection and model optimisation presented in the reviewed literature and it closes the gap identified by Yu et al. (2024) in the case of Pi-class constrained hardware IDS..

2.7 Related Works

This section summarizes five currently published studies that are most relevant to the project, all using lightweight ML-based IDS and running on Raspberry Pi hardware. Each study is discussed regarding the method, findings and weaknesses of that study. The section ends with a comparison table of all five works and the common limitation in each of them is highlighted, and this will be the specific research gap that this project addresses.

2.7.1 Lightweight IDS in IoT Networks Using Raspberry Pi 3B+

Kuthada et al. (2021) proposed a lightweight anomaly-based IDS targeting DoS attack detection in IoT networks, implemented on a Raspberry Pi 3B+. The motivation for the work was the unsuitability of conventional IDS solutions for resource-constrained IoT nodes, which operate under strict limits on processing power and memory. The authors addressed this through a

combination of Correlation-based Feature Selection (CFS) and a Support Vector Machine (SVM) classifier with an RBF kernel.

The UNSW-NB15 dataset was used for training and evaluation, containing 175,341 instances across nine attack categories. CFS reduced the original 44 features to just three — source-to-destination transaction bytes (sbytes), packet arrival rate, and source-to-destination time-to-live (sttl). This reduction was necessary because attempting to train the SVM on all 44 features caused the Raspberry Pi 3B+ to crash due to insufficient memory, confirming that feature reduction is a hard deployment requirement rather than an optional optimisation on this hardware class.

With CFS applied, the model achieved 98% accuracy, 96% recall, and a false positive rate of 0.02 on the DoS test subset. Without CFS, accuracy on a laptop was 94% but recall dropped sharply to 79%, demonstrating that the reduced feature set actually improved generalisation in addition to enabling deployment. The authors concluded that CFS produced a lightweight model without compromising detection performance.

A key limitation acknowledged by the authors is that the system was only evaluated for DoS attacks. The study does not report performance on other attack classes in the UNSW-NB15 dataset, does not evaluate live traffic capture, and uses a fixed classification threshold throughout. No adaptive mechanism of any kind is present in the system design.

2.7.2 Energy Consumption of On-Device Machine Learning Models for IoT Intrusion Detection

For this purpose, the authors of this study, Tekin et al. (2023) compared the energy consumption of six ML algorithms: Logistic Regression (LR), k-Nearest Neighbour (k-NN), Decision Tree (DT), Random Forest (RF), Naive Bayes (NB), and Artificial Neural Network (ANN) on three platforms: cloud (Azure), edge (Dell laptop), and IoT device (Raspberry Pi 4 with 8 GB RAM). The data set employed was the DS2OS smart home traffic dataset with 357,953 samples in 7 attack categories.

The study was conducted on the training phase and the inference phase separately. The authors determined that RF needed 624.43 seconds to train on the Raspberry Pi at the highest number of samples (3.5 million), while DT took 40.86 seconds. This is because RF needs to process multiple trees in parallel, and multicore processors in cloud and edge environments can greatly help here, but are very limited on the Pi. For this reason, training of RF is not energy efficient, the authors explicitly say.

For inference, DT achieved the best result at 65.06 microseconds on the Azure IoT Kit end device, because tree-based algorithms perform computation without any multiplication operations. RF inference on the Pi ranged from approximately 10 to 18 microseconds depending on dataset size, which is still practical for real-time classification. In terms of accuracy, DT, RF, k-NN, and ANN all achieved 99% accuracy on the dataset, confirming that RF is not uniquely superior in accuracy but carries a significantly higher energy cost for training.

The paper's contribution to this project is its empirical confirmation that the offline-train, online-infer paradigm is the only energy-viable deployment model for RF on Pi-class hardware. No dynamic threshold mechanism or adaptive detection behaviour is present in this work.

2.7.3 Optimized Ensemble Deep Learning for Real-Time Intrusion Detection on Resource-Constrained Raspberry Pi Devices

Musthafa et al. (2025) built upon their earlier stacked LSTM model by applying a dual-stage compression pipeline to make it deployable on the Raspberry Pi 3B+, which has only 1 GB of RAM. The original model, trained on the UNSW-NB15 dataset using ANOVA feature selection to produce a refined set of 36 features, had achieved 97.46% accuracy but had a model size of 2,581 KB and was too large to run on the Pi without optimisation.

The compression pipeline involved converting the model to TensorFlow Lite format, applying post-training quantisation to reduce weight precision from 32-bit floating point to INT8, and applying magnitude pruning using a PolynomialDecay schedule with final sparsity of 0.5. The combined optimised model (OT) reduced model size by 90.9% from 2,581 KB to 235 KB, loading time by 96.64% to 223.8 ms, and inference time by 99.25% to 2.4 ms, while maintaining 97.26% accuracy.

To evaluate the system under realistic conditions, the authors used an Apache Kafka-based testbed that simulated variable traffic loads using multiple producers representing concurrent attackers, with controlled transmission delays. The system remained stable under dynamic load with accuracy of 97.3% in all nine experimental scenarios which is independent of the number of producers and simulation delay. Under these scenarios, CPU usage reached 111.8% indicating the Pi 3B+ makes good use of its multicore processing power.

The authors note that one of the important things that they have left out is the online model updating. They see this as a compromise: learning online with Pi-class devices is risky due to performance drift, it needs labelled data, and it puts additional computational burden on the device that it can't handle. The same threshold was used for all detection in the study. No adaptive threshold mechanism was developed and tested.

2.7.4 Design and Implementation of a Machine Learning-Based Adaptive IDS on Raspberry Pi for Smart Home Network Security

Adrian et al. (2025) developed and deployed an IDS system on the Raspberry Pi platform for smart home networks, which combines network traffic monitoring, iptables-based firewall rules, whitelist-based access control, and Elastic Stack for logging and visualisation of network traffic in real time. The system was tested with simulated DDoS, brute force and malware injection attacks.

A neural network was selected as the classifier as it performed best in the model selection process for detecting complex attack patterns, outperforming Decision Trees and Support Vector Machines. According to the paper, the detection accuracy is 85% as compared to 65% for traditional rule-based approaches, while the response time is 2 seconds as compared to 5 seconds for the baseline. The authors call the system adaptive because it is able to continuously learn by adjusting the detection models as new traffic patterns are observed from the continuous network traffic.

It should be noted that the "adaptive" behaviour mentioned here is periodic model retraining and not any dynamic change of the detection threshold. The system uses the same decision boundary during its operation, and the adaptation is performed at the model level through retraining the classifier with new data. This is a very different mechanism than one that changes at inference time based on the traffic conditions.

There are a number of limitations that are self-evident from the paper itself: It does not contain a confusion matrix, precision-recall analysis or ROC/AUC values, making the results difficult to compare with other studies. The structure of neural networks and the size of the model are not given in detail. No specific Raspberry Pi model is mentioned other than the general platform description.

2.7.5 A Cost-Effective Machine Learning Based Network Intrusion Detection System Using Raspberry Pi for Real-Time Analysis

Wijethilaka et al. (2025) proposed a NIDS based on Raspberry Pi 4 for small and medium businesses, which was trained and tested on NF-UQ-NIDS dataset and implemented with Random Forest classifier. It covers a wide variety of attack types such as DoS, DDoS, SQL injection, port scanning, malware, exploits and reconnaissance, and also has attributes like incoming packet count, outgoing packet count, TCP flags, flow duration, protocol identifier and more. The authors note that SMOTE was used when training the data, as the authors note that this helped to greatly reduce the false positive rate for the final model, given the significant class imbalance in the data set.

To reduce the dimensionality and to find the most discriminative features for classification, feature selection was done using PCA and RFE. When capturing live traffic during operation, the system employed pyshark and tcpdump to process the incoming traffic streams on the Raspberry Pi 4. The deployed model has a detection accuracy of 96.5% and an average response time of 50 milliseconds. To put this into perspective, the authors compared their system to the signature-based systems Snort, Suricata, Palo Alto Firewalls, and Cisco Firepower, finding that their system had a comparable accuracy to the signature-based systems, and outperformed them in detecting zero-day threats because of their ML-based approach.

Another physical alarm system was also included using LEDs and buzzer attached to the Pi's GPIO pins and email notifications using the Python smtplib library, to give instant multi-channel alarm. The authors intentionally set it up as a viable cybersecurity offering for the SMB market, which is not being well-served by enterprise-grade products.

One of the major drawbacks of this work is the fact that the classification threshold is set and then kept constant during deployment. Although the authors showed good performance in a controlled environment, there is no mechanism to set the detection boundary according to the change in traffic volume or baseline behaviour. This is a common issue among all of the implementations reviewed and it is this very problem that the adaptive threshold mechanism in this project aims to solve.

2.7.6 Summary of Related Works

Across all the 5 studied literatures, comparison between each of them are listed below in Table 2.3:

Table 2.3: Summary of Related Works

RESEARCH WORK	ALGORITHM	PLATFORM	DATASET	KEY RESULTS	LIMITATION
Lightweight IDS in IoT Networks Using Raspberry Pi 3B+ (Kuthada et al., 2021)	SVM, with CFS feature selection	Raspberry Pi 3B+	UNSW-NB15	98% accuracy, 0.02 FPR with 3 selected features; DoS detection only	Fixed threshold; single attack type evaluated; no live traffic capture
Energy Consumption of On-Device ML Models for IoT Intrusion Detection (Tekin et al., 2023)	LR, k-NN, DT, RF, NB, ANN compared	Raspberry Pi 4 (8 GB), Azure Cloud, Dell Laptop	DS2OS smart home dataset	DT most energy-efficient (65.06 µs inference); RF training on Pi takes 624.43s; all top models achieve 99% accuracy	Fixed threshold; energy study only, no live IDS deployment; no adaptive mechanism
Optimized Ensemble Deep Learning for Real-Time Intrusion Detection on Raspberry Pi (Musthafa et al., 2025)	Stacked LSTM with ANOVA feature selection, pruning and quantisation	Raspberry Pi 3B+ (1 GB RAM)	UNSW-NB15	90.9% model size reduction (2,581 KB to 235 KB); 97.3% accuracy maintained; 2.4 ms inference; tested under variable Kafka traffic loads	Fixed threshold; online model updating deliberately excluded; limited to UNSW-NB15 dataset
Design and Implementation of a Machine Learning-Based Adaptive IDS on Raspberry Pi for Smart Home Network Security (Adrian et al., 2025)	Neural network (architecture unspecified) with Elastic Stack integration	Raspberry Pi (model unspecified)	Simulated smart home traffic (DDoS, brute force, malware injection)	85% accuracy vs 65% for rule-based baseline; 2s response time vs 5s baseline; iptables and whitelist integration	Fixed threshold; "adaptive" refers to model retraining, not threshold adjustment; no confusion matrix, ROC, or AUC reported; neural network architecture not specified
A Cost-Effective ML-Based NIDS Using Raspberry Pi for Real-Time Analysis (Wijethilaka et al., 2025)	Random Forest with PCA and RFE feature selection, SMOTE for class balancing	Raspberry Pi 4	NF-UQ-NIDS	96.5% accuracy; 50ms average response time; significantly reduced FPR after SMOTE; LED, buzzer, and email alerting	Fixed threshold; no dynamic or adaptive threshold mechanism; evaluated under controlled conditions only

CHAPTER 3

METHODOLOGY

3.1 Introduction

The methodology to design, build and test Adaptive Threshold Intrusion Detection System with Random Forest on Raspberry Pi is explained in this chapter. Research methodology is a systematic approach that has been used to answer a research question, which includes processes, methods, and tools used in a project. The strategy followed here was based on the nature of the problem: a working prototype of an IDS should be developed, implemented on low cost hardware, and evaluated against specific performance goals under normal traffic and attack traffic loads.

Waterfall model is chosen as the methodology. This model divides development process into a series of phases which are executed sequentially and each phase is completed before the next one begins. This is appropriate for the project as the requirements are known beforehand before any development takes place. The hardware is known (Raspberry Pi), the machine learning algorithm is motivated by literature (Random Forest), the dataset is already selected (CIC-IDS2017) and the evaluation metrics are predetermined, including detection rate, false positive rate, CPU and memory consumption, and response time. The core design does not require modification during the course of the project, so Waterfall is more suitable than iterative approaches, which are used for projects where requirements may evolve as the project is developed (Petersen et al., 2009).

The methodology is broken down into four stages: Planning, Design, Development and Evaluation. Planning includes algorithm selection, tool selection, hardware selection, and datasets. Design translates those decisions into system diagrams, network topology and detailed module specifications. Development includes setting up the environment, training the model offline, deployment to the Raspberry Pi, and creating the adaptive threshold mechanism and the Flask dashboard. Evaluation is used to test the completed system with simulated normal and attack traffic and compares the performance with the project goals.

3.2 Overview of Methodology

This section is an explanation of the method selected for the project management and project structuring. It explains and justifies the choice of the Waterfall model due to the type of project requirements, outlines the three key principles of the model, and outlines the overall Waterfall model for the four phases of the project. There is also a summary table of tasks and deliverables for each phase.

3.2.1 Waterfall Methodology

The Waterfall model was chosen as the project management methodology for this project. It structures development into a series of stages in which one stage has to be finished before the next can start (Petersen et al., 2009). The three basic concepts of this model are: The model is

sequential, End user's participation in the development process is minimal, Requirements, development process and the outcome is fully documented.

There are three reasons why this methodology is appropriate. First of all, the project has a clear set of requirements. Prior to development, the problem, objectives, scope, hardware, algorithm, and evaluation criteria are all determined. Waterfall works best when there is a lack of change in project goals, meaning that if project goals change, it is as if the process has to be repeated from the beginning of the project (Bassil, 2012). Second, the project is time-bound, with a fixed duration of 28 weeks, and has fixed milestones for each phase, just like Waterfall. Thirdly, sufficient time is allowed in the early stages to allow for good planning and design before any implementation takes place. Adequate upfront planning is one of the critical conditions for Waterfall to be effective, as the remainder of the process relies upon the completeness of the requirements collected during the initial phase. Bassil (2012) states that all other phases of Waterfall depend on the completeness of the requirements collected during the initial phase.

The Waterfall model is a five-step process model that includes requirements, design, implementation, verification, and maintenance (Bassil, 2012). In this project they are broken down into four stages: Planning, Design, Development, and Evaluation. Documentation is not limited to the end of the project, but is used throughout the project..

3.2.2 Project Workflow Diagram

The project follows the Waterfall sequence shown in Figure 3.1. Planning feeds into Design, which feeds into Development, which feeds into Evaluation. Each phase has a defined set of tasks and deliverables, summarised in Table 4 below.

Table 3.1: Tasks and Deliverables of Each Phases

PHASE	TASKS	DELIVERABLES
Planning	1. Determine problem statement 2. Conduct literature review 3. Select dataset, algorithm, and tools 4. Identify hardware and software requirements	1. Problem statement, objectives, and project scope. 2. Selected dataset (CIC-IDS2017) and algorithm (Random Forest) 3. Hardware and software specifications 4. https://www.kaggle.com/datasets/dhoogla/cicids2017
Design	1. Design system architecture 2. Design network testbed topology 3. Design traffic capture and feature extraction pipeline 4. Design adaptive threshold (EMA) mechanism 5. Design Flask dashboard interface	1. System architecture diagram 2. Network topology diagram 3. Feature extraction design 4. EMA threshold design 5. UI wireframes
Development	1. Configure Raspberry Pi OS 2. Set up Python environment	1. Trained Random Forest model

	3. Preprocess dataset and train Random Forest model offline 4. Deploy model to Raspberry Pi 5. Implement live traffic capture and feature extraction 6. Implement adaptive threshold 7. Develop Flask dashboard	2. Deployed IDS on Raspberry Pi 3. Working adaptive threshold module 4. Functional web dashboard
Evaluation	1. Simulate attack traffic using hping3 and nmap 2. Measure detection rate 3. Measure false positive rate under fixed and adaptive threshold 4. Measure CPU and memory utilisation 5. Measure system response latency	1. Experimental results comparing adaptive vs. fixed threshold performance

3.3 Planning

All decisions prior to starting design or development work are covered in this section. It sets up the system architecture conceptually, justifies the selection of the data set, the machine learning algorithm and the adaptive threshold mechanism, and outlines the hardware and software requirements for the project. All decisions made here are carried over to the Design and Development phase.

3.3.1 System Architecture Overview

The system is designed as a network-based IDS (NIDS) with the Raspberry Pi being a gateway between a test laptop and a router, all network traffic goes through it for monitoring. The decision to adopt this architecture stemmed from the need for complete visibility of the flow level of the entire network, without the need to install any software on the individual devices that are connected to the network (Wijethilaka et al., 2025).

The system consists of three layers. The first layer is the traffic capture layer in which the live network packets are captured on the Raspberry Pi using the tcpdump and are aggregated into 5-second flow windows per source IP. The second one is the detection layer, where 12 features are extracted from each flow window and sent to the trained Random Forest classifier, which provides a threat score. That score is then compared against the adaptive threshold computed by the EMA mechanism to determine whether an alert should be raised. The third is the presentation layer, where alerts and traffic logs are stored in a local SQLite database and displayed on a Flask web dashboard accessible through a browser on the local network.

The detailed visual representation of this architecture, including the specific components and data flow within each layer, is presented in the Design phase in Section 3.4.1.

3.3.2 Dataset Selection (CIC-IDS2017)

Four benchmark datasets were evaluated for this project: CIC-IDS2017, UNSW-NB15, NSL-KDD, and KDD99. The comparison is summarised in Table 3.2 below. The most critical selection criterion for this project is training-deployment consistency: the feature extraction tool used to build the dataset must be the same tool used to extract features from live traffic on the Raspberry Pi. A mismatch between the two means that the same statistical feature is computed differently at training time and at inference time, which degrades real-world performance regardless of how well the model performs on the offline test set (Thakkar and Lohiya, 2020).

Table 3.2: Comparison between different datasets

Dataset	Year	Attack Types Covered	Feature Extraction Tool	Realistic Traffic Generation	Training-Deployment Consistency	Citation
CIC-IDS2017	2017	DoS, DDoS, Brute Force, Port Scan, Botnet, Web Attacks	CICFlowMeter	Yes (B-Profile system)	High (CICFlowMeter used in live deployment)	Thakkar & Lohiya (2020); Sharafaldin et al. (2018)
UNSW-NB15	2015	DoS, Reconnaissance, Fuzzers, Exploits, Backdoors, Shellcode	Argus, Bro-IDS	Partial (IXIA tool)	Low (different extraction tools)	Kuthada et al. (2021); Musthafa et al. (2025)
NSL-KDD	2009	DoS, Probe, R2L, U2R	Pre-extracted (no tool)	No (derived from KDD99)	Very Low (pre-extracted features)	Thakkar & Lohiya (2020)
KDD99	1999	DoS, Probe, R2L, U2R	Pre-extracted (no tool)	No (simulated, outdated)	Very Low (pre-extracted features)	Thakkar & Lohiya (2020)

CIC-IDS2017 is selected because it is the only dataset built with CICFlowMeter, the same tool used for live feature extraction on the Raspberry Pi in this project. This ensures that the 80 flow-level features in the training data are computed by identical logic to the features extracted from real traffic, directly reducing distributional mismatch (Sharafaldin et al., 2018). The dataset also covers all three attack types targeted in this project, namely DoS, ICMP flood, and port scanning, and was generated using realistic user behaviour profiling rather than simple packet simulation.

UNSW-NB15 is a credible dataset but its features are extracted using Argus and Bro-IDS, neither of which is compatible with CICFlowMeter, making it unsuitable for live deployment consistency. NSL-KDD and KDD99 are both excluded on the grounds of age and pre-extracted features that cannot be reproduced by any current tool, ruling out any live deployment pipeline.

From the full 80-feature space of CIC-IDS2017, 12 features were selected based on their documented discriminative importance for the three target attack types, coverage of volume, flag, and entropy characteristics, and the inference efficiency requirement on constrained hardware confirmed by Tekin et al. (2023). Kuthada et al. (2021) and Wijethilaka et al. (2025) both identify packet arrival rate, byte counts, TCP flags, and flow duration as among the most critical features for lightweight Pi-based IDS, which form the core of the selected subset.

3.3.3 Machine Learning Algorithm Selection (Random Forest)

This project used a suitable machine learning algorithm selected from five candidate classifiers that are commonly used in network intrusion detection research; namely Random Forest, Decision Tree, Support Vector Machine (SVM), k-Nearest Neighbour (k-NN) and Naive Bayes. The comparison was done on four parameters that are directly relevant to the constraints of this project: Detection accuracy, False positive rate, Inference speed, Suitability for deployment on constrained hardware like Raspberry Pi 4.

Decision Tree is the least complex of all the classifiers scientifically. Tekin et al. (2023) computed the inference time for a Raspberry Pi to be 624.43 seconds for Random Forest, versus 40.86 seconds for Decision Tree, and noted sub-microsecond inference latency per flow. However, a single decision tree has a tendency to overfit the data since it constructs one deterministic path through the training data without any variance reduction mechanism. When compared with Random Forest, Decision Tree has consistently lower accuracy and recall in the same circumstances, especially in minority attack classes, and has a higher false positive rate on imbalanced network traffic data (Samantaray et al., 2024). The marginal speed up of marginal inference of Decision Tree compared to the tradeoff in accuracy and false positive is not worth it for this project, particularly as both algorithms' inference latency is within the 100-millisecond target on Pi hardware.

IDS has been studied extensively using SVM, which can realize competitive accuracy on well-balanced data set. But SVM classification involves kernel computations that demand a huge amount of memory and processing time at the time of inference, especially when using a non-linear kernel like Radial Basis Function (RBF) kernel. Classifiers compared directly on Raspberry Pi-class platforms have demonstrated that the inference latency of SVM is significantly higher than that of tree based classifiers, and that SVM without extra pre-processing on class imbalance is not as effective as the tree based classifiers (Samantaray et al., 2024). With the UNSW-NB15 dataset, Kuthada et al. (2021) obtained 98% accuracy after reducing the number of features to three due to an entire lack of memory for the Raspberry Pi 3B+ when training with SVM on 44 features. This hardware incompatibility at scale will not make SVM desirable for this project since there are 12 features needed for concurrent live traffic windows.

k-NN is a simple method that—inferring about each instance by comparing to all the instances in the training set—scaling linearly with the number of instances in the training set. This instance-based computation is impractical to implement on limited hardware. However, Samantaray et al., (2024) found that the latency of inference with k-NN on any IoT-class device is significantly high compared to tree-based classifiers and the memory overhead of storing the entire training set in RAM is impractical for devices with limited RAM, such as the Raspberry Pi 3B+ and 4. In addition to hardware limitations, another challenge with k-NN is its poor performance on high dimensional imbalanced data sets if it is not tuned carefully for the distance measure, as lower F1-scores were observed than other ensemble methods in standard IDS benchmark (Samantaray et al., 2024).

Naive Bayes is one of the cheapest of the five options and can be used in very low resource settings. It relies on the important and usually unrealistic assumption of conditional independence of all features, however, which is not valid in network traffic data because features like packet count, byte count and flow duration are naturally correlated. This assumption has a negative impact on the calibration of Network traffic classification tasks.

Comparative studies have revealed that Naive Bayes achieved the highest number of false positive and the lowest overall accuracy of the five classifiers evaluated in this study, especially in detecting DoS and port scan attacks, where features are correlated to distinguish attack traffic from normal traffic (Samantaray et al., 2024; Tekin et al., 2023).

Random forest overcomes the problem of overfitting in Decision Tree by creating a forest of decision trees on random random forest samples of the training data, and then fusing the predictions from the different decision trees with the majority vote. This bagging technique helps to decrease variance and improve generalisation over a single tree, and also preserves the inference benefits of tree-based computation as there is no need for any multiplication operations during inference, making the classification cost low and predictable on the Raspberry Pi (Tekin et al., 2023). In this project, the native feature importance scores provided by Random Forest are obtained via the Gini impurity method and are used to confirm that the 12 features used for this project are the ones that will be used for live deployment. Random Forest has consistently achieved the highest accuracy and the lowest false positive rate over the other five candidate algorithms across several comparative studies on the IDS benchmarks, and especially on imbalanced data, combined with SMOTE class balancing during training (Samantaray et al., 2024; Wijethilaka et al., 2025).

The results of the five algorithms are summarized below in Table 3.3. From this analysis, Random Forest is chosen as the classifier to be used in this project. The cost of training a RF model is much higher than that of a Decision Tree model, but the offline-train, online-infer deployment mode means that the cost is fully borne on the laptop training machine and the model is only deployed to the Raspberry Pi for inference by passing through the serialised model. This approach is energy-viable and has already been shown to deliver real-time performance on Pi hardware by Wijethilaka et al. (2025), who achieved 96.5% accuracy of detection and an average latency of 50 milliseconds when running a Random Forest on the Pi hardware used in this work.

Table 3.3: Comparison of Machine Learning Algorithms for IDS on Constrained Hardware

Algorithm	Accuracy	Precision	Recall	F1-Score	Inference Speed	Suitability for Constrained Hardware
Random Forest	High (96-99%)	High	High	High	Low-Medium (no multiplications)	High
Decision Tree	High (97-99%)	Medium-High	Medium-High	Medium-High	Lowest (single tree)	Very High
SVM	Medium-High (94-97%)	Medium-High	Medium	Medium	High (kernel computations)	Low
k-NN	Medium (90-95%)	Medium	Medium	Medium	Very High (instance-based)	Very Low
Naive Bayes	Low-Medium (80-90%)	Low-Medium	Medium	Low-Medium	Lowest	High

3.3.4 Adaptive Threshold Mechanism Design (EMA)

The unique contribution in this project is the adaptive threshold mechanism. The system is not based on a set detection threshold, but on a rolling estimate of network behaviour, which is continually adjusted during operation. This will allow the threshold to increase when there is heavy traffic on the network (which should be the case), thereby reducing false positives, and to decrease when the network is quiet, thereby increasing sensitivity to subtle attacks (Yu et al., 2024).

This rolling baseline will be computed using Exponentially Weighted Moving Average (EMA). EMA gives exponentially decreasing weight to older observations, thus recent traffic behaviour has greater influence on the baseline than older traffic behaviour. This allows the baseline to be responsive to actual changes in traffic patterns without being affected by short-term fluctuations that would otherwise bias a simple average (Yu et al., 2024).

In the Design phase (Section 3.4.5), the detailed formulation of the EMA update rules, computation of the threshold, and the decision flow for raising alerts are provided.

3.3.5 Hardware Specifications (Raspberry Pi)

The Raspberry Pi 4 Model B with 4 GB of RAM was selected as the deployment platform for this project. It serves as the network gateway through which all monitored traffic passes. The Raspberry Pi 4 was chosen over the 3B+ because it provides a more capable ARM Cortex-A72 processor and a larger RAM capacity, which gives more headroom for running the traffic capture process, the classifier, the EMA computation, the SQLite database, and the Flask dashboard concurrently without hitting the memory limits documented on the 3B+ by Musthafa et al. (2025).

The full hardware specifications for the development and deployment device are listed in Table 3.4 below.

Table 3.4: Raspberry Pi Hardware Specifications

HARDWARE	DESCRIPTION
Device	Raspberry Pi 4 Model B
Processor	1.5 GHz (1.8 Ghz boosted) ARM Cortex-A72 (quad-core, 64-bit)
RAM	4 GB LPDDR4
Storage	32 GB microSD card
Operating System	Raspberry Pi OS (64-bit, Debian-based)
Network Interface	Built-in Gigabit Ethernet + USB Wi-Fi adapter
Power Supply	Raspberry Pi USB-C power adapter (5V, 3A)

3.3.6 Software and Tools

The software and tools selected for this project are listed in Table 3.5 below. Each tool was chosen based on its documented suitability for Python-based IDS development on constrained hardware and its availability under open-source licensing.

Table 3.5: Software and Tools Used

NO.	ITEM	TOOL/VERSION	PURPOSE
1.	Operating System	Raspberry Pi OS 64-bit	Base operating system for deployment device
2.	Programming Language	Python 3.10	Core development language for all modules
3.	Traffic Capture	tcpdump	Live packet capture on the Raspberry Pi
4.	Feature Extraction	CICFlowMeter	Extract flow-level features from captured packets
5.	Machine Learning	scikit-learn (Random Forest)	Offline model training and inference
6.	Adaptive Threshold	Custom Python module	EMA-based threshold computation
7.	Database	SQLite	Local storage of alert logs and traffic records
8.	Web Dashboard	Flask	Browser-accessible alert and monitoring interface
9.	Attack Simulation	hping3, nmap	Simulate DoS, ICMP flood, and port scan attacks
10.	Development IDE	Visual Studio Code	Code writing and debugging on development laptop
11.	Documentation	Microsoft Word, Microsoft Excel	Project report and Gantt chart
12.	Design Tool	draw.io & Visual Paradigm Online Drawing Tool	System diagrams and flowcharts

3.3.7 Project Timeline

Table 3.6: Gantt Chart of Project Timeline

PHASE	DELIVERY TIME (WEEK)													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
PLANNING														
Finalize Project Topic														
Discuss with Supervisor														
Identify Research Problem														
Create Proposal														
DESIGN														
System architecture and network topology														
Feature extraction and EMA threshold design														
Flask dashboard UI wireframe														
IP auto-block design														
DEVELOPMENT														
Environment setup (Raspberry Pi OS, Python, SQLite)														
Dataset preprocessing and model training														
Live traffic capture and adaptive threshold implementation														
Flask dashboard and IP auto-block implementation														
TESTING														
Testbed setup and attack simulation														
Detection rate and false positive rate experiments														
CPU, memory, and latency experiments														
Results analysis and final report compilation														
DOCUMENTATION														

3.4 Design

The planning decisions are translated into concrete designs of the system in this section. It shows the diagram of system architecture, network testbed topology, feature extraction pipeline, training process of Random Forest, two-tier EMA adaptive threshold process including learning window phase, alert design, dashboard design, and IP auto-block mitigation design. A diagram or table is provided to assist with the development of each subsection.

3.4.1 System Overall Flowchart

The overall operational flow of the system, from start-up to shut-down, is shown in Figure 3.1. The gateway is initialised when the program is started, the Random Forest model is loaded and the baseline tracker runs a 10 minute learning window while the Flask dashboard is running. Threat score (TS) is not alerted during this time and is added to a running mean and standard deviation.Threat score (TS) is not alerted during this time and added to a running mean and standard deviation. Upon completion, the system enters into live detection mode.

For live detection, raw packets are continually collected and evaluated for successful feature extraction, and windows that fail or don't contain enough packets are discarded. Features that have been successfully extracted are passed to the Random Forest model to get a probability score (0.5-1.0) and then compared with the current adaptive threshold. Any score lower than the threshold is recorded as normal, then a new threshold is calculated every 5 minutes before the baseline is changed. If IP auto-block is turned on, the source IP is blocked using iptables before the baseline is updated; if IP auto-block is not turned on, the baseline is updated directly, and a dashboard alert is triggered if there is an attack. In both, the dashboard refreshes and the system looks for a shutdown signal: If a signal is not received, it loops back to get the next window; otherwise, it saves the state and disables the dashboard.

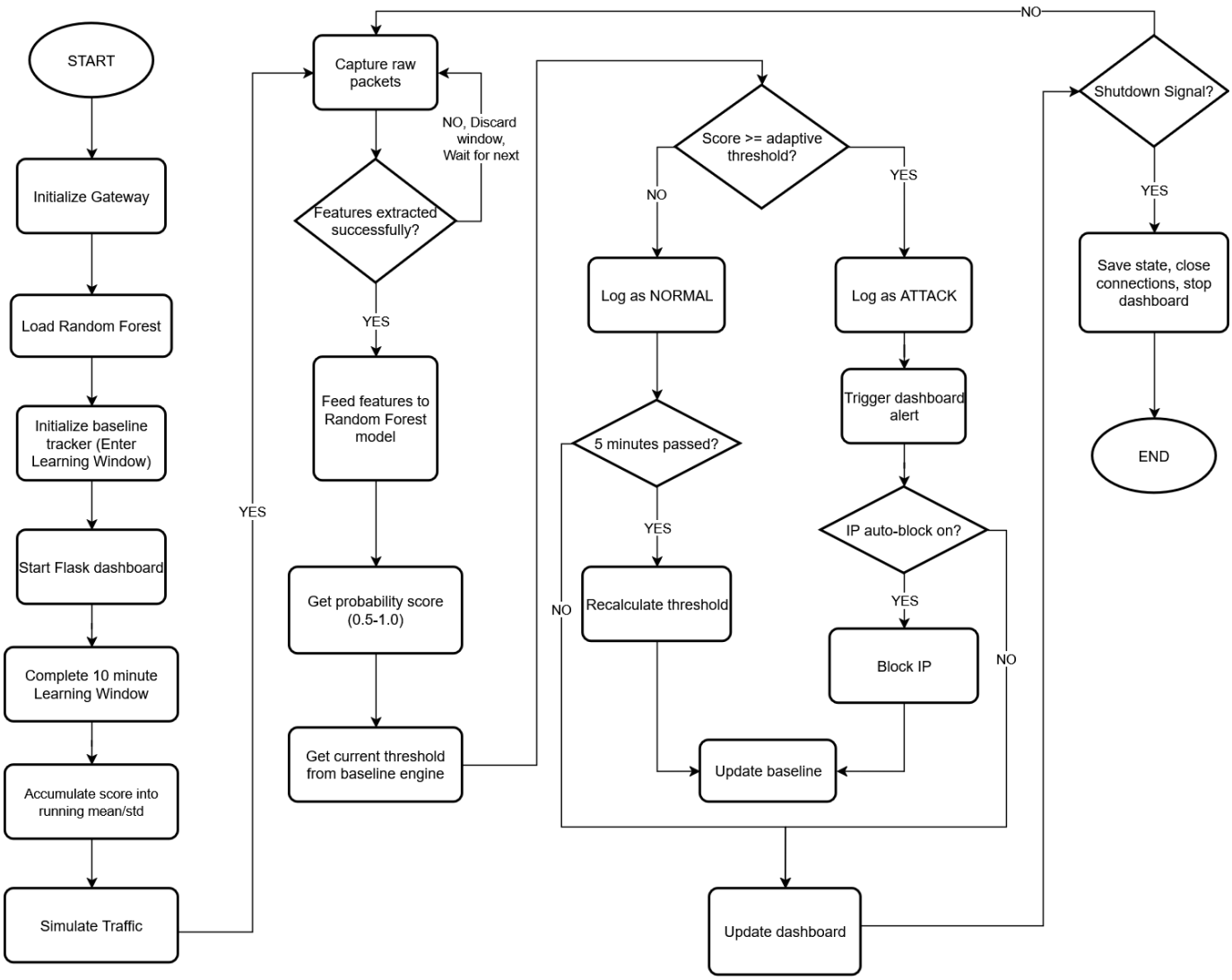


Figure 3.1: Overall System Flowchart

3.4.2 System Architecture Diagram

The system architecture follows a three-layer design as established in the Planning phase. Figure 3.1 shows how data flows through the system from raw packet capture to alert generation and dashboard display.

The traffic capture layer sits at the network interface of the Raspberry Pi. All packets passing through the device are captured by tcpdump and grouped into 5-second flow windows per source IP address. The detection layer receives these flow windows, extracts the 12 selected features, passes them to the trained Random Forest classifier, and compares the resulting threat score against the EMA-computed adaptive threshold to decide whether an alert should be raised. The presentation layer stores all alerts and traffic records in a local SQLite database and makes them visible through the Flask web dashboard, which can be accessed from any browser on the local network.

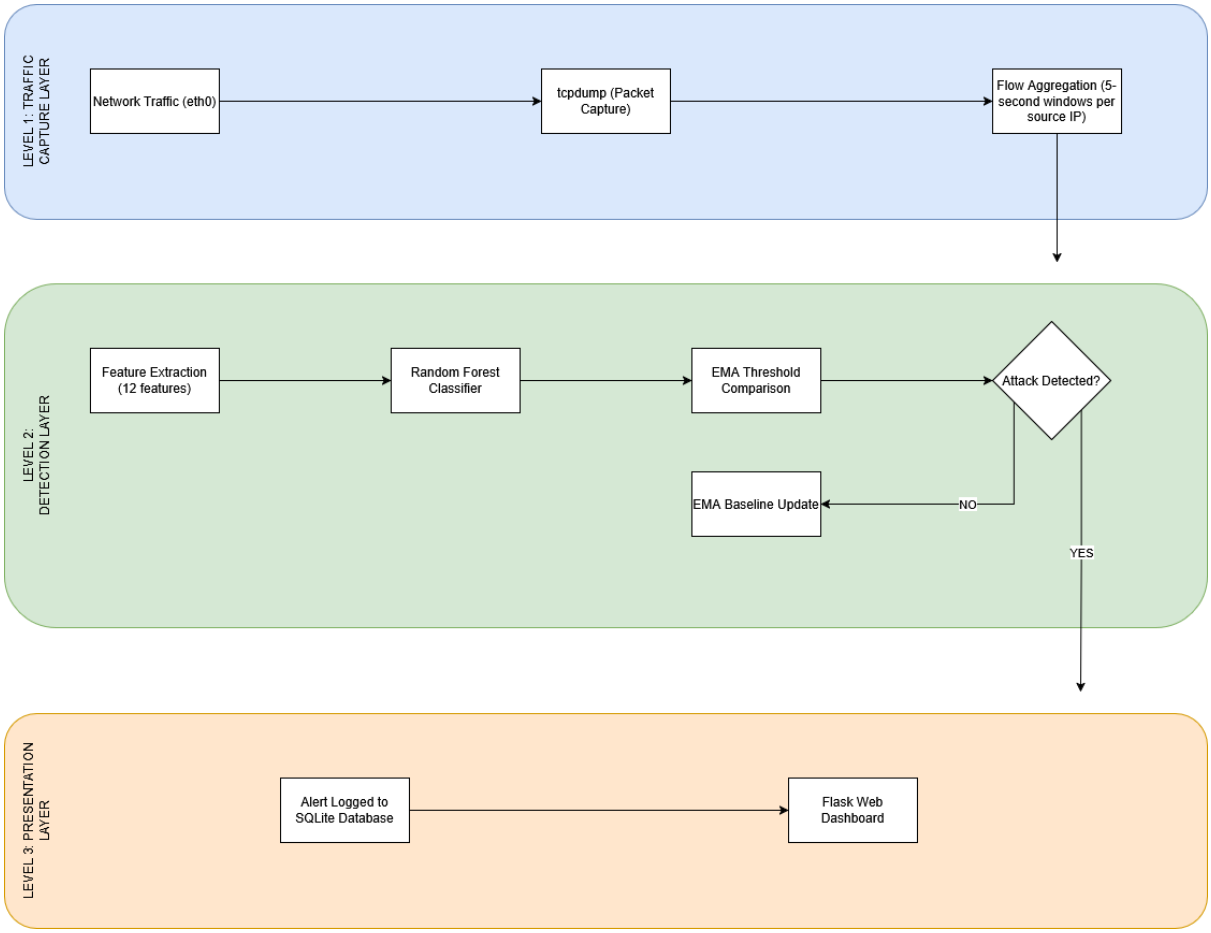


Figure 3.2: System Architecture Diagram

3.4.3 Network Topology / Testbed Setup

The testbed is designed to represent the home or small office network environment in a controlled lab environment. The Raspberry Pi is used as a gateway between a test laptop and a router, meaning that any traffic that comes out of the test laptop has to pass through the Pi and then out to the router and onto the network. This placement will ensure that the IDS can see all traffic being monitored.

There are three nodes in the network. The attack machine is a laptop running the Kali Linux operating system, with a set of simulated attack traffic created with the use of hping3 and nmap. The monitoring device is the Raspberry Pi 4, which monitors all the traffic using it and then runs the classifier, applies the adaptive threshold and serves the Flask dashboard. The target machine is a second machine on the network, receiving simulated attack traffic. Both machines are connected to the Raspberry Pi using its network interfaces, one of which is facing the attacker while the other is facing the router and target machine.

This deployment configuration, depicted in Figure 3.2, is similar to that of Wijethilaka et al. (2025), which involved using the Raspberry Pi as a gateway device to provide full flow-level visibility without endpoint agents.

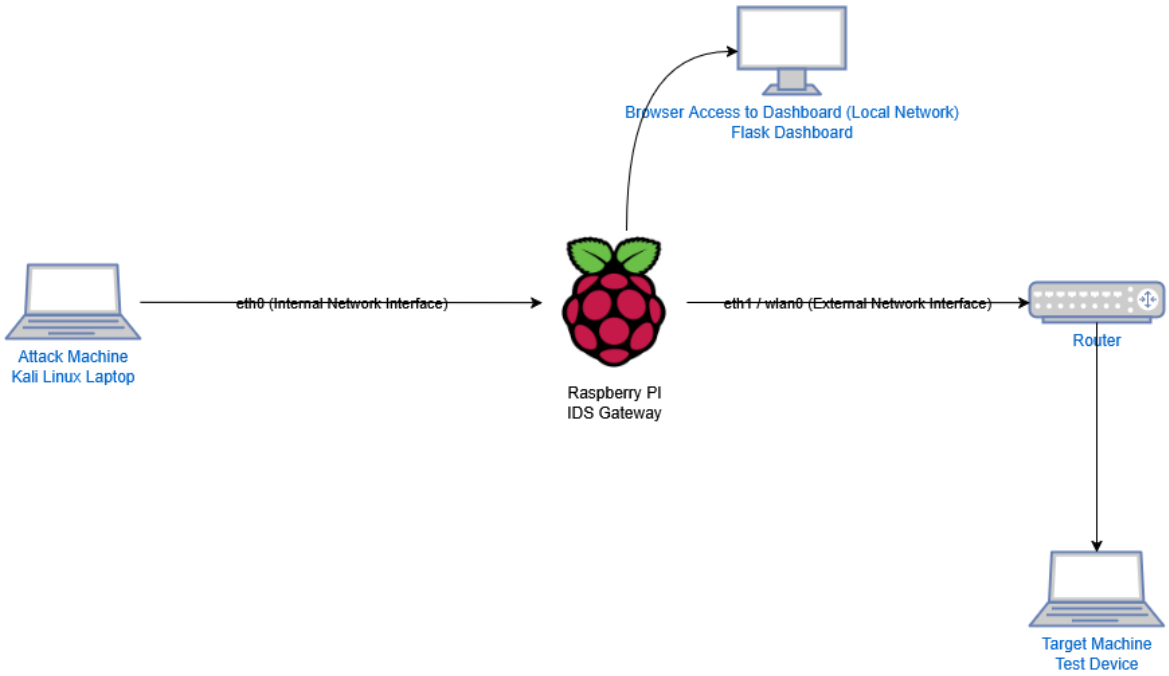


Figure 3.3: Network Topology/Testbed Setup

3.4.4 Traffic Capture and Feature Extraction Design

A tcpdump program is used on the Raspberry Pi's network interface to capture traffic. Packets are continually captured and are organized into 5-second flow windows for each source IP address. A flow window gathers all packets sent from a particular source IP into this window, and performs statistical analysis over the packets to generate the 12 input features used by the classifier.

The 12 features were chosen from the feature space known from the CIC-IDS2017, because they have been identified as relevant to DoS, ICMP flood and port scan attacks. According to Kuthada et al. (2021) some of the most discriminative features for lightweight IDS on constrained hardware are packet arrival rate, and the number of bytes. Wijethilaka et al. (2025) have shown that the most critical features to recognize attack patterns are TCP flags, flow duration, and number of incoming and outgoing packets. The features selected are shown in Table 3.7 below.

Table 3.7: Features Extracted from Traffic

NO.	FEATURE	DESCRIPTION
1.	Flow Duration	Total duration of the flow window in microseconds
2.	Total Forward Packets	Number of packets sent from source to destination
3.	Total Backward Packets	Number of packets sent from destination to source

4.	Total Length of Forward Packets	Total bytes sent in the forward direction
5.	Total Length of Backward Packets	Total bytes sent in the backward direction
6.	Forward Packet Length Mean	Average size of forward packets in bytes
7.	Backward Packet Length Mean	Average size of backward packets in bytes
8.	Flow Packets per Second	Total packets divided by flow duration
9.	Flow Bytes per Second	Total bytes divided by flow duration
10.	SYN Flag Count	Number of packets with the SYN flag set
11.	RST Flag Count	Number of packets with the RST flag set
12.	Destination Port Entropy	Measure of spread across destination ports contacted

3.4.5 Random Forest Model Design

The model is fully developed offline on a regular development laptop and is deployed to the Raspberry Pi. The offline-train, online-infer approach is required as the training of the model directly on the Pi is too expensive for practical use, as demonstrated by Tekin et al., (2023) who measured 624.43 seconds for the Random Forest model to be trained at smaller sizes of data.

There are four steps in the training pipeline. The CIC-IDS2017 dataset is loaded and filtered to keep only 12 features and the class label, as follows. The dataset is then split into 80% training data and 20% test data using stratified sampling to ensure both splits contain proportional representation of normal and attack classes. Second, infinite and null values occur in the dataset are removed, feature values are normalised, and class distribution is balanced using SMOTE to avoid biased classification by the classifier toward the majority normal class as demonstrated by Wijethilaka et al. (2025). Third, the model is fitted with scikit-learn RandomForestClassifier using Cross Validation with parameters optimised. Fourth, the trained model is exported as a serialised file using joblib and sent to the Raspberry Pi for inference.

In live operation on the Raspberry Pi, each flow window of 5 seconds is fed into the same feature extraction logic as during training and the loaded model gives a threat score for each flow window. The results of the scoring process are then sent to the adaptive threshold module, which is described in the following section. Figure 3.3 shows the illustration for Random Forest Training Pipeline.

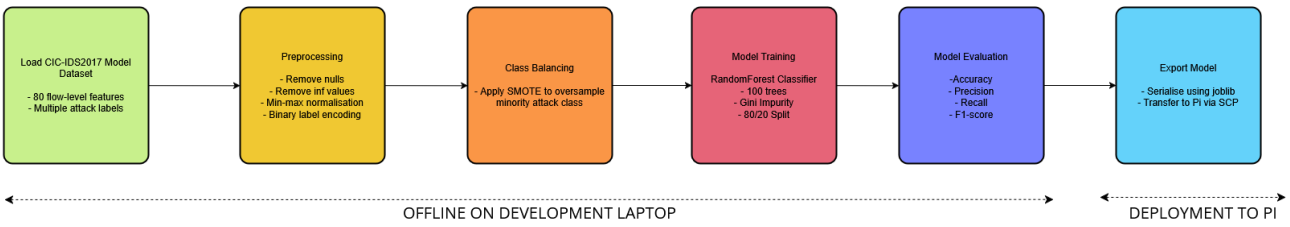


Figure 3.4: Random Forest Training Pipeline

3.4.6 Adaptive Threshold (EMA) Design

The adaptive threshold mechanism operates in two stages: an initial baseline learning phase that runs once at system startup, and a live operation phase that runs continuously afterward. This separation exists because the EMA mechanism requires a meaningful starting baseline before it can make any detection decisions. Without it, the first few flow windows would have nothing valid to compare against.

Baseline Learning Phase

Once the system begins, it goes into a learning window where all traffic is treated as normal and no decisions are made on traffic detection. For this entire time the Random Forest model continues to score each incoming flow window but is not responsible for alert generation. The scores are only used for calculating an initial baseline mean and standard deviation only, not to generate alerts. The length of the learning window is determined at 10 minutes, which was selected as being long enough to cover a representative sample of typical traffic patterns for several flow windows but short enough that the total delay for the system deployment is not significant.

The advantage of the approach is that it is known to be limited in that it assumes that the network is not attacked during the learning window. If an attack does happen in this time frame, the resulting baseline would be off and the threshold calculated from it would be dubious from the beginning. It is recognized as a shortcoming of the self-supervised baselining approach, and is overcome during evaluation by operating the learning window under known and normal traffic conditions, as detailed in Section 3.4.2 in describing the controlled testbed.

When the learning window is finished, the system immediately switches to live detection with the baseline values derived from this phase.

Live Operation Phase

The adaptive threshold mechanism changes two values during the live operation at different rates. After each flow window of 5 seconds, the mean and standard deviation of the baseline are updated, as follows:

mean_new = (beta x score) + ((1 - beta) x mean_old)

std_new = sqrt(beta x (score - mean_new)² + (1 - beta) x std_old²)

This is a real-time per window update which keeps the baseline up to date with the latest traffic behaviour. The threshold value is actually the decision boundary used when compared to the incoming scores; however the threshold value is only recalculated once every 5 minutes:

threshold = baseline_mean + (k x baseline_std)

The two-tier design is similar to that of Yu et al. (2024), which calculates dynamic thresholds, instead of continuously, with fixed time windows. The decision boundary does not shift with each window, as this would cause an inconsistent score between successive windows which would be scored against slightly different thresholds, when there wasn't any meaningful changes in network conditions. The interval of 5 minutes is long enough to dampen out background noise, yet short enough to capture real changes in traffic conditions, like the beginning of a backup job or a video call.

Every time a new threat score is generated, it is checked against the active threat score, which is the one that is calculated at the latest 5 minute threat score recalculating point. If the score is above this limit, then the flow is considered an attack and the system will proceed to alert and logging. The score is not deemed to exceed the threshold, and no change in this threshold will be made until the next 5-minute recalculation, if the score is below the threshold, the flow is treated as normal traffic, and only the baseline mean and standard deviation are updated.

It is also important to note that the threshold is not set to reactively become more restrictive during an actual attack. It is not for monitoring drift of legitimate traffic during an attack, but rather drift over time. The threshold adjusting during the attack window is not responsible for the sensitivity to the attacks, but rather the score of the Random Forest classifier. The adaptive threshold flow is plotted in Figure 3.4.

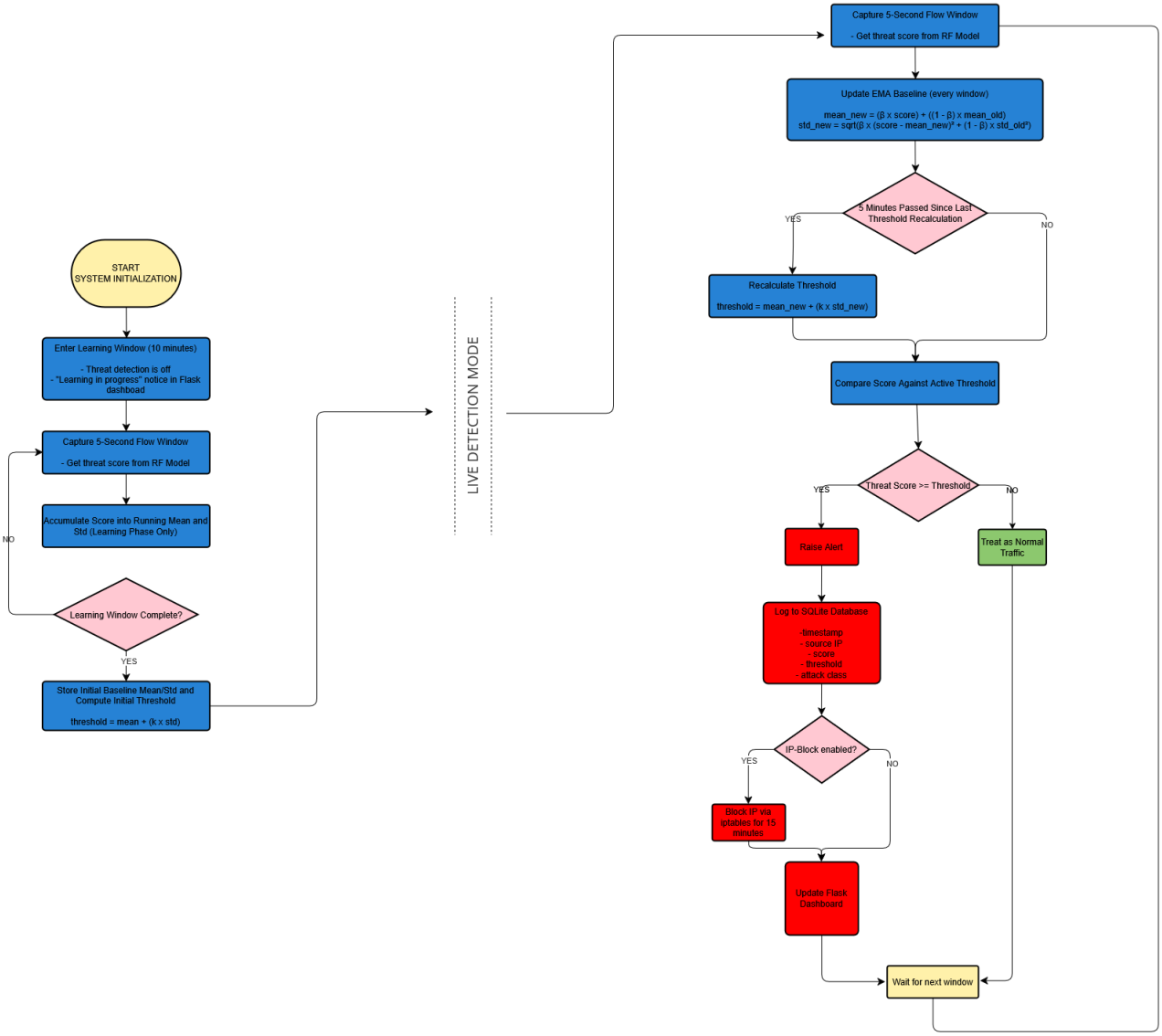


Figure 3.5: EMA Adaptive Threshold Flow Diagram

3.4.7 Database Design

There are three related tables in the database, each with one view on the dashboard: traffic_log, alerts, and blocked_ips. The traffic_log table logs each flow window that was processed, whether the processor succeeded or failed, and stores a variety of fields, including log_id (the primary key), total_packets, flow_duration, threshold_at_time, threat_score, source_ip, and timestamp. It is related to alerts with a "may produce" relationship, because there are flow windows that are greater than the adaptive threshold that produce an alert. Each alerts record has alert_id as its primary key, which is linked back to log_id by a foreign key, and also stores source_ip, threat_score, threshold_at_time and attack_class, so that the live alert view can trace each alert back to its source traffic log record.

The blocked_ips table is used to support the optional IP auto-block mitigation feature (Section 3.4.8) and has a “may result in” relationship with alerts; that is, IP blocks are not automatic consequences of all alerts. The primary key is block_id, and it has a foreign key constraint on alert_id, as well as source_ip, block_start, and status to monitor if a block is active, expired or manually removed. The structure is chained, as is the three-layer system architecture in Section 3.4.1, with the idea of preserving a chain of evidence from the raw traffic data to the detection data to the mitigation data, all of which are kept in light tables to fit the limited resources available on the Raspberry Pi.

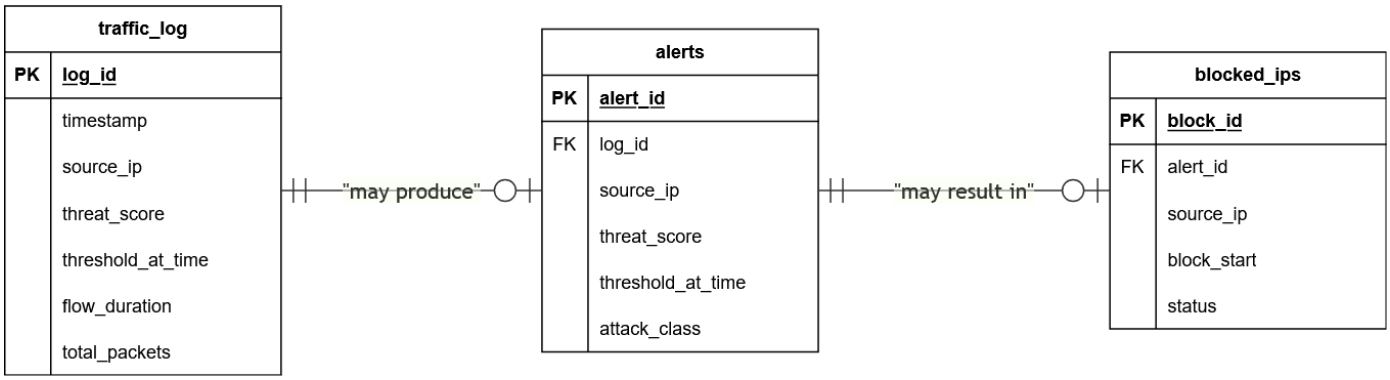


Figure 3.6: Database Design

3.4.8 IP Auto-Block (Mitigation) Design

The system not only detects the attack, it also has an optional feature to mitigate it, which automatically denies access to the source IP of the traffic identified as an attack. The purpose of this feature is not to provide the main contribution of the project, but to further enhance the usefulness of the detection mechanism.

It is not permanently activated, but rather is an option on the Flask setting that can be disabled or enabled as needed. This is intentional as a design decision in the context of the false positive issue covered in this project. If the network access were to be denied because an IP address was automatically blocked due to a mis-classification, then the whole point of the adaptive threshold is to reduce false positives over a fixed threshold is lost. It is optional to enable auto-block, so that users can turn it off while conducting the evaluation experiments in Section 3.6, which are focused on detection and FPR measurements, but not on blocking behaviour.

If auto-block is turned "on", then the mitigation logic is activated after an alert is raised. DPIP filtrates the source IP address of the flagged flow, which is subsequently blocked by iptables rules directly applied on the Raspberry Pi, similar to the approach that Adrian et al. (2025) take for firewall-based mitigation on IDS deployments on Pi-class machines. It is not a permanent block, it will be applied for a specified time of 15 minutes then the rule is removed automatically and the IP address can communicate normally again. A fixed-duration block was selected instead of permanent block to minimise the impact of a potential false positive, as a permanent block would cause an extended denial of access if a genuine device was incorrectly identified.

The blocked IP addresses are stored and displayed on the dashboard with the time of the block and the time when it expires, so that the user can unblock an IP address before the time limit if he recognizes it as a legitimate device.

Additionally, there is a toggle to enable or disable the IP auto-block feature, and a dedicated panel displaying the list of currently blocked IP addresses, their start time of the block and their scheduled end time of the block. This enables the user to track and (if required) manually override the mitigation behaviour defined in Section 3.4.7 directly in the live alert view.

3.4.9 Alert and Dashboard (Flask) Design

A web dashboard (Flask) is available for system activity monitoring from a web browser. It runs locally on the Raspberry Pi, and is available for any device on the same network by typing the Pi's IP address and port into a browser.

The dashboard is designed around three main views. The page of the live alert in Figure 3.5 shows in real-time all incoming alerts, including the classification of the attack type, the source IP, the current threshold value, and the threat score of the threat being detected. Figure 3.6 displays the traffic log view page that displays a historical view of all traffic processed by the system, including normal and flagged traffic. The user can get an overview of the current CPU and memory usage and the current EMA baseline values from the system status view page shown in Figure 3.7, and thus be made aware of the health and sensitivity of the running system.

All of the alerts and traffic records are stored in a local sqlite database on the Raspberry Pi. SQLite was selected due to its minimal memory usage, its requirement for no server process and its ability to cope with the amount of data produced by a single gateway IDS in a home or small office environment (Musthafa et al., 2025).

The dashboard also has a switch to turn the IP auto-block feature off and on, and a panel to display the list of IPs currently blocked, the time they are set to be blocked and the time they are scheduled to be unblocked. This enables the user to see the mitigation behaviour described in Section 3.4.7 and enable manual override of the behaviour in real-time from the live alert view if required.

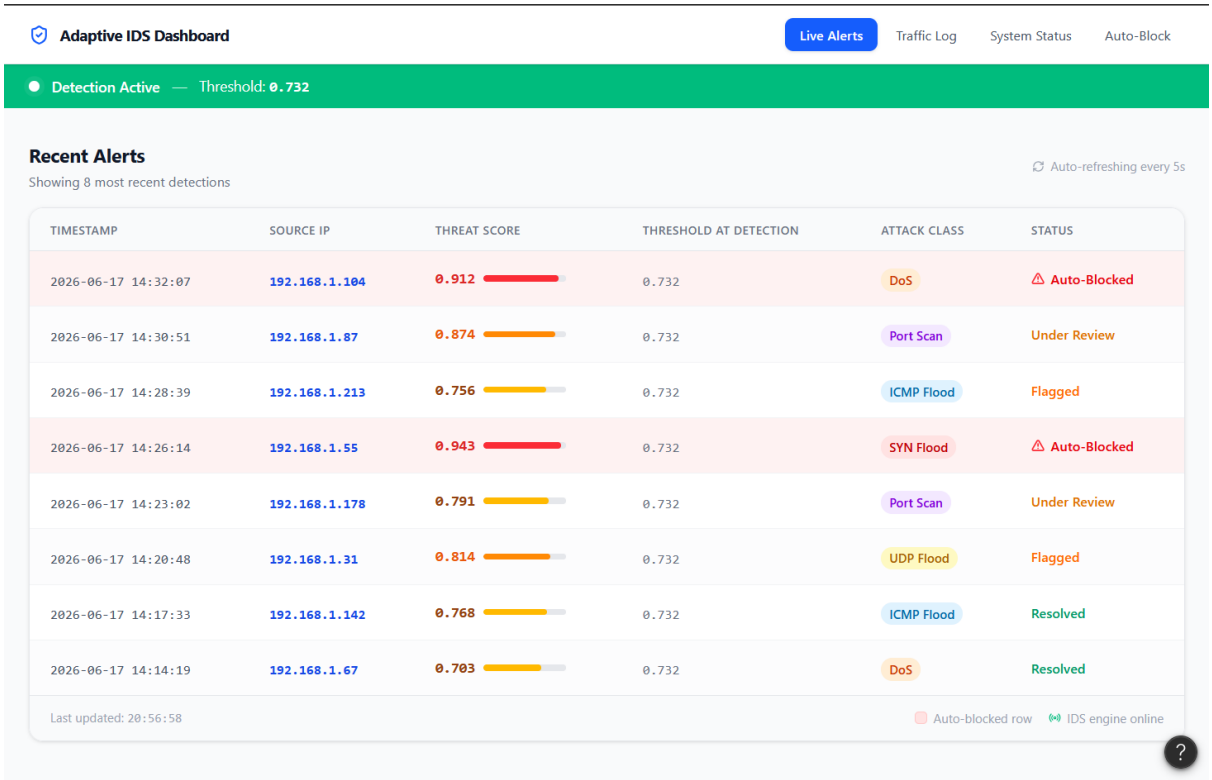


Figure 3.7: User Interface of Live Alert Page

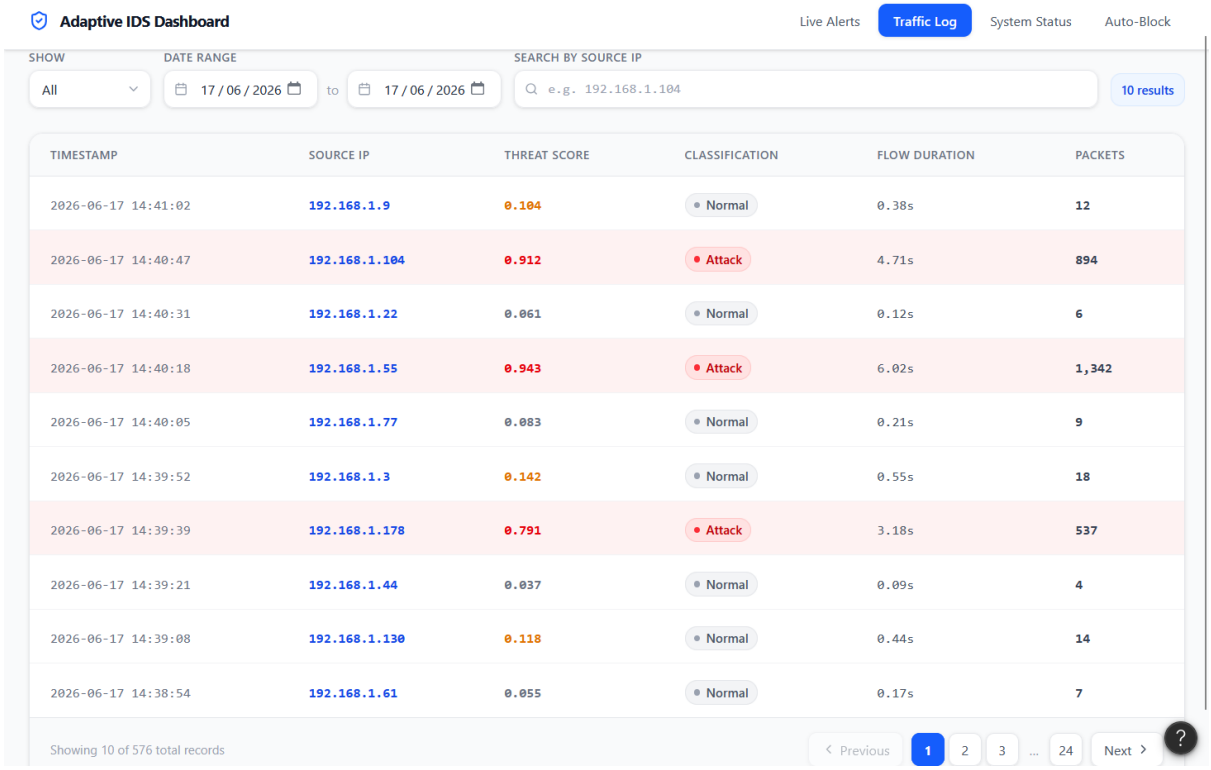


Figure 3.8: User Interface of Traffic Log View Page

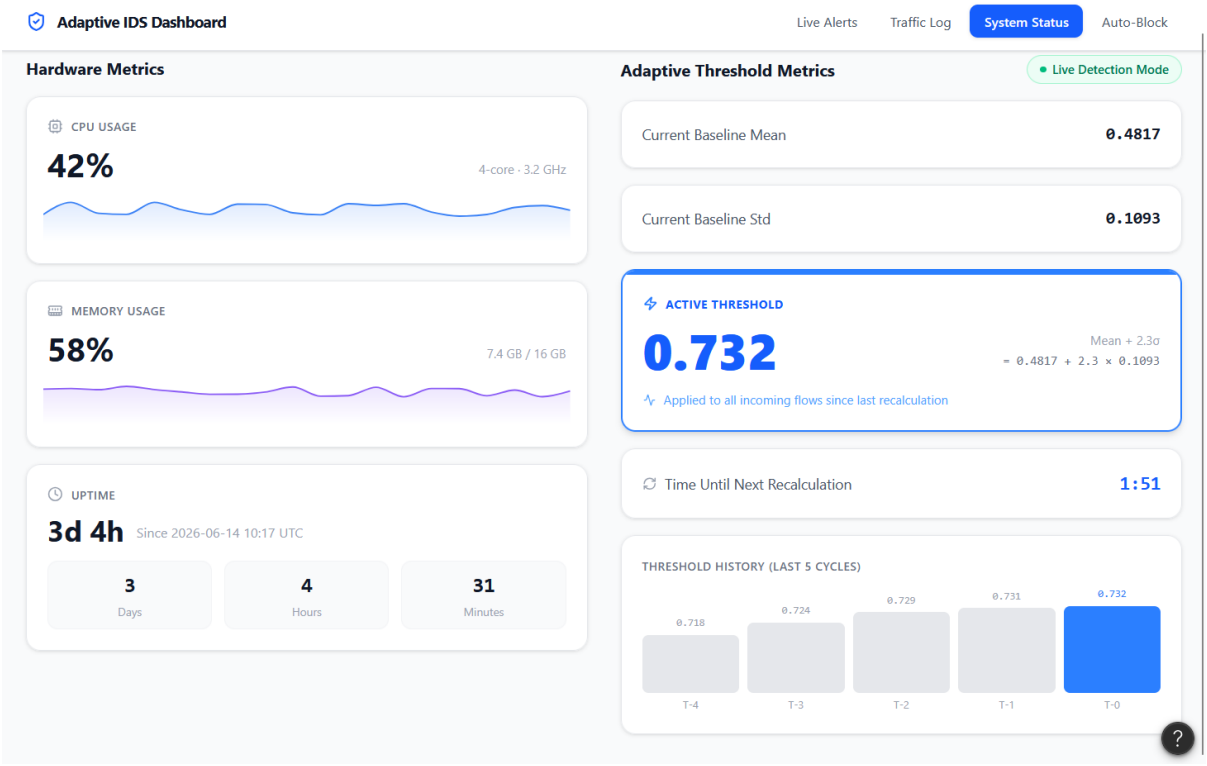


Figure 3.9: User Interface of System Status View Page

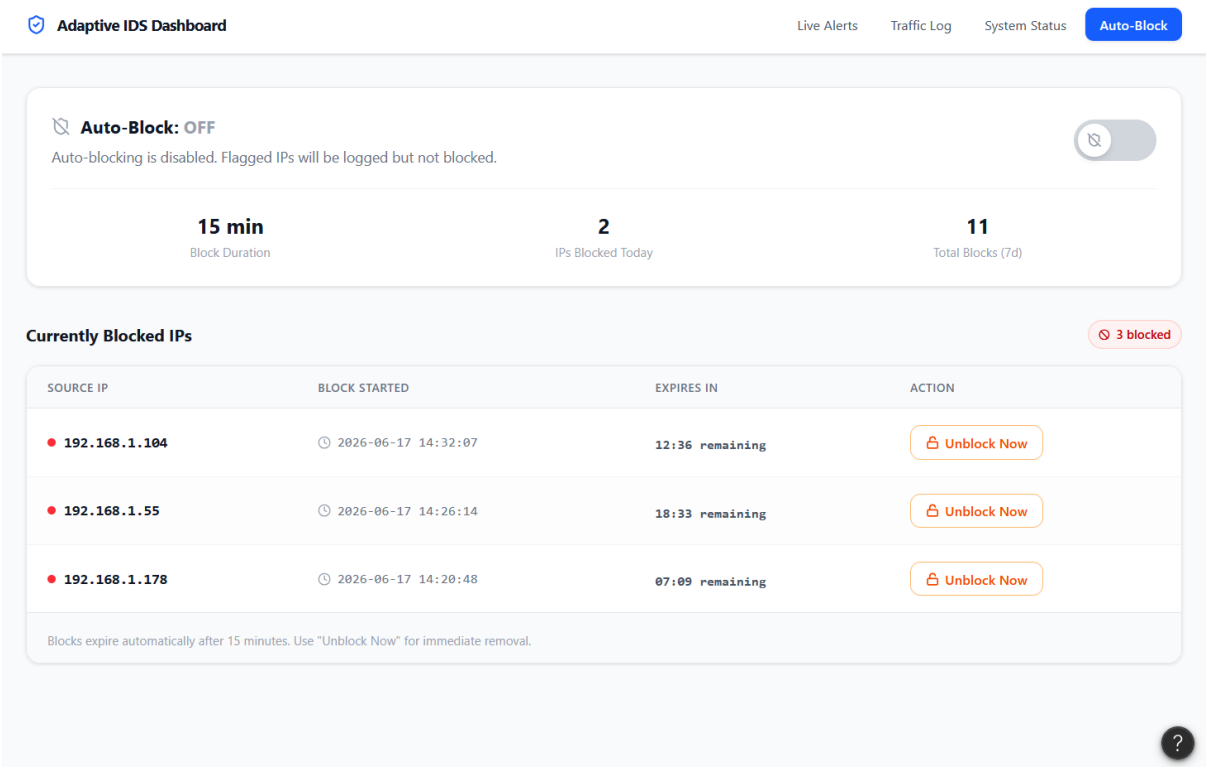


Figure 3.10: User Interface of IP Auto-Block Panel Page

3.5 Development

This section outlines the practical implementation of each of the design elements of Section 3.4. It includes environment configuration of Raspberry Pi, preprocessing of the offline dataset, training of the model, deployment of the trained model, capturing and extracting the traffic data in real-time, the adaptive threshold module, the Flask web dashboard, and the IP auto-block module. The specifics of implementing the design presented in Section 3.4 are written in such a way that they are consistent with those design decisions.

3.5.1 Environment Setup

3.5.1.1 Raspberry Pi OS Configuration

The Raspberry Pi 4 is configured with the 64-bit version of Raspberry Pi OS, which is a Debian-based Linux distribution officially supported by the Raspberry Pi Foundation. The 64-bit version is chosen over the 32-bit version because it allows the operating system and running processes to address more than 4 GB of memory, which is necessary to run the traffic capture process, the classifier, the EMA module, the SQLite database, and the Flask dashboard concurrently without memory allocation errors.

After flashing the OS image to the microSD card, the network interfaces are configured to allow the Pi to sit between the attack machine and the router. One interface is assigned to face the internal test network and the other faces the router. IP forwarding is enabled in the kernel so that packets arriving on one interface are forwarded to the other, allowing the Pi to function as a gateway while simultaneously capturing all passing traffic for analysis.

3.5.1.2 Python Environment and Dependencies

All system modules are written in Python 3.10. A virtual environment is created on the Raspberry Pi to isolate the project dependencies from the system Python installation. The required packages are installed within this virtual environment using pip. The core dependencies are listed in Table 3.8 below.

Table 3.8: Core Python Dependencies Used

NO.	PACKAGE	VERSION	PURPOSE
1.	scikit-learn	1.3.0	Loading and running the trained Random Forest model
2.	pandas	2.0.3	Data handling during feature extraction
3.	numpy	1.24.0	Numerical computation for EMA updates
4.	joblib	1.3.0	Loading the serialised Random Forest model file
5.	flask	3.0.0	Web dashboard server

6.	imbalanced-learn	0.11.0	SMOTE for class balancing during offline training
7.	scapy	2.5.0	Packet parsing during feature extraction

3.5.1.3 SQLite Database Setup

The SQLite database is created locally on the Raspberry Pi to store alert logs and traffic records. Two tables are created within the database. The first table, named alerts, stores every flow that was flagged as an attack, recording the timestamp, source IP address, threat score, threshold value at the time of detection, and the predicted attack class. The second table, named traffic_log, stores a summary record for every processed flow window regardless of whether it was flagged, allowing the user to review normal traffic history alongside alerts through the dashboard.

SQLite was selected because it operates as a file-based database with no separate server process required, has a minimal memory footprint suitable for the Raspberry Pi, and provides sufficient read and write performance for the data volumes generated by a single-gateway IDS in a home or small office environment (Musthafa et al., 2025).

3.5.2 Dataset Pre-processing and Feature Selection

During the offline phase of the dataset preprocessing, it is done on the development laptop prior to the model training. The dataset used for CIC-IDS2017 is loaded from its CSV files and filtered to include only the 12 selected features and the class label column of the dataset. There are four steps in the preprocessing pipeline.

The first step is to delete rows with ‘infinite' values or empty cells. The CIC-IDS2017 dataset is also known to have a few rows with infinite values (e.g. flow bytes per second) due to flow duration being zero in these rows (Thakkar & Lohiya, 2020). These rows are deleted instead of being added back into the training set, so as not to add spurious values to the training set.

Second, features are normalized by min-max scaling to scale all features between 0 and 1. This will ensure that features like total bytes do not influence features like counts of flags during tree splitting because they have very different ranges.

Third, the classes are reduced to just two: normal and attack. All types of attacks, such as DoS, DDoS, port scan and brute force, with attack labels are collected under one attack label. This binary classification is in line with the project goal, which is to identify attacks in general and not the exact attack type.

Fourth, SMOTE is used to oversample the minority classes of the attack to balance the class distribution when applied to the training split. Wijethilaka et al. (2025) found that the training when performed with SMOTE additive rule led to a significantly lower false positive rate because it did not push the classifier towards the majority normal class.

3.5.3 Random Forest Model Training (Offline)

Training the Random Forest model is done offline on a development laptop with the RandomForestClassifier of scikit-learn. The preprocessed data set is partitioned using stratified sampling into 80% training data and 20% test data, having equal proportion of normal and attack classes in both partitions.

The following are the key hyperparameters: The number of estimator trees will be fixed at 100, this is a reasonable number of trees to have to balance the time required to make inferences on the Raspberry Pi against the number of trees that the ensemble should have to avoid too much inference overhead. The depth of each tree is not limited in the initial training, and is optimized using cross validation if overfitting is detected. The number of samples to split the internal node is 2, and Gini impurity is the criterion used for the quality of the split, which was indicated to be good for tasks such as network traffic classification (Wijethilaka et al., 2025).

Once trained, the model is tested on a held-out test set to make sure that its behavior in the offline setting is suitable for deployment. At this stage the accuracy, precision, recall and f1 score are recorded. The trained model is then serialised using joblib and transferred to the Raspberry Pi using secure copy (SCP) to deploy it.

3.5.4 Model Deployment on Raspberry Pi

The serialised model file is transferred to the Raspberry Pi and loaded into memory at system startup using joblib. A Python script running as a background service handles model loading, traffic capture coordination, feature extraction, classification, and threshold management. This script is configured to start automatically when the Raspberry Pi boots using a systemd service unit, so the IDS begins monitoring immediately without requiring manual intervention each time the device is powered on.

The loaded model occupies a fixed portion of the Pi's RAM throughout operation. Based on the findings of Musthafa et al. (2025), who achieved successful deployment on a Raspberry Pi 3B+ with only 1 GB of RAM after model compression, a scikit-learn Random Forest model with 100 trees trained on 12 features is expected to remain well within the 4 GB RAM available on the Raspberry Pi 4 used in this project without requiring any compression or quantisation.

3.5.5 Live Traffic Capture and Feature Extraction

Live traffic capture is performed by tcpdump running on the Raspberry Pi network interface facing the internal test network. tcpdump is configured to capture all packets passing through the interface and write them to a rolling buffer. A Python script reads from this buffer continuously, parses each packet using scapy, and assigns it to a flow window based on its source IP address and the current 5-second interval.

At the end of each 5-second window, the script computes the 12 features listed in Table 3.1 from the packets collected for each source IP during that interval. Flow windows containing fewer than five packets are discarded to avoid classifying incomplete or idle flows that would not carry enough information for reliable detection. The computed feature vector for each flow window is then passed directly to the loaded Random Forest model for classification.

This capture and extraction approach is consistent with the method used by Wijethilaka et al. (2025), who processed live packet streams on a Raspberry Pi 4 using tcpdump and reported an average classification response time of 50 milliseconds, confirming that this pipeline is feasible within the real-time constraints of the deployment environment.

3.5.6 Adaptive Threshold Implementation

The adaptive threshold mechanism is realized in the form of a Python class with the following instance variables: current mean, current standard deviation, and the threshold value. The class is in two modes, the first is a learning mode and the second is a live detection mode; both of these modes are described in Section 3.4.5.

Learning Window Implementation

At start-up, the class is placed in learning mode and a 10 minute timer is activated. In this time, each flow window that is passed to the Random Forest model is passed through the baseline learning function that adds up the threat score and calculates a running mean and standard deviation. This phase cannot be compared to a threshold and cannot alert. Within this time frame, the Flask dashboard will show "Learning in Progress" to indicate that detection is not active.

When the 10 minute timer runs out the mean and standard deviation of the accumulated data is saved as the starting baseline and the class moves into live detection mode. At this transition point, the first threshold value is computed as it is in live operation, therefore, a valid value is used as a threshold value and not a default or arbitrary threshold value.

Live Detection Implementation

The class carries out two updates on different schedules, corresponding to the two-tier design discussed in Section 3.4.5, in the live detection mode.

Once every 5 second flow window, the class is given a new threat score, and the mean and standard deviation are updated with the EMA formulas. The update occurs without any condition, on all windows, as the baseline has to be always monitoring traffic behaviour.

There is also a 5-minute timer for each window that runs independent of the per-window updates and that triggers a threshold recalculation. At the end of this timer, the current baseline mean and standard deviation are recorded, a new threshold is calculated and stored as the active threshold. This value will stay constant and will be compared to in every subsequent 5 minute timer.

The class compares the incoming threat score with the existing threshold, rather than a new one just calculated. When the score is above the active threshold, the class will return an alert flag, the score, the value of the active threshold at the time of detection, and the predicted attack class. If the score is below the threshold, a normal flag of the class is returned and only the baseline update described above is done.

The duration of the learning window, the 5-minute interval for recalculating the sensitivity coefficient k and the smoothing coefficient beta are all set as configurable parameters at the startup of system and are not hardcoded. This makes it possible to adjust these values without

changing the implementation logic, useful when comparing with baseline values in Section 3.6.3 which are fixed for a threshold value.

3.5.7 IP Auto-Block Implementation

The IP auto-block feature is implemented as a module that runs only when the corresponding setting is enabled on the Flask dashboard. When an alert is raised by the adaptive threshold module described in Section 3.5.6, the system checks whether auto-block is currently enabled before taking any mitigation action.

If auto-block is enabled, the module executes an iptables command to drop all incoming traffic from the flagged source IP address. The block entry, including the source IP, the timestamp it was applied, and the calculated expiry time 15 minutes later, is recorded in a dedicated table in the SQLite database, separate from the alerts and traffic_log tables described in Section 3.5.1.3. A background thread checks this table at regular intervals and removes any iptables rule whose expiry time has passed, restoring normal access for that IP address automatically.

The Flask dashboard reads from this table to display the currently blocked IP addresses and their remaining block duration, as described in Section 3.4.6. If the user manually unblocks an IP address through the dashboard before its scheduled expiry, the corresponding iptables rule is removed immediately and the database entry is updated to reflect the early removal.

This feature is disabled by default at system startup and must be explicitly enabled through the dashboard, consistent with the design rationale discussed in Section 3.4.6.

3.5.8 Web Dashboard Development (Flask)

The Flask web dashboard is written as a simple python web app that is running on the Raspberry Pi local machine. It reads live alerts and traffic data from sqlite data base and displays it in three HTML pages based on the three views that were designed during the Design phase: live alerts, traffic log, system status.

The live alert view connects to the alerts table in the database and fetches the latest alerted flows and updates it every 5 seconds with a JavaScript timer, showing the latest flows in a table. The following shows the time stamp, source IP address, threat score, current threshold value at detection time, and predicted attack class. The traffic log view will provide a paginated view of all flow windows that have been processed in the traffic_log table, including both normal and flagged views. The system status view shows the current CPU and memory usage of the Raspberry Pi using the python psutil library, as well as the live baseline mean, std and threshold to allow the user to see the adaptive threshold in real-time.

The dashboard can be accessed from any device on the network by going to the IP address of the raspberry pi and the port on which flask is running. It does not require any external Internet connection and all data is stored locally on the device, which meets the requirements of privacy and self-contained operation of a home network IDS.

3.6 Evaluation and Testing

In this section, the process for testing and evaluating the completed system when it is finished is described. It includes the testbed configuration, the attack commands used to create test traffic, and the four experiments measuring detection rate, false positive rate, CPU and memory utilisation, and system response latency. These experiments will be repeated to assess the validity of using the adaptive threshold mechanism to perform better than a fixed threshold approach under the same conditions.

3.6.1 Testbed Configuration and Attack Simulation (hping3, nmap)

The evaluation is conducted in a controlled lab environment using the testbed described in Section 3.4.2. The Raspberry Pi 4 is positioned as the gateway between the attack machine and the target machine. All traffic generated during testing passes through the Pi, allowing the IDS to capture and classify every flow produced during both normal and attack scenarios.

Two tools are used to simulate attack traffic. hping3 is used to simulate DoS and ICMP flood attacks, and nmap is used to simulate port scan attacks. The specific commands used for each attack type are listed in Table 3.9 below.

Table 3.9: Tools and Commands for Attacks Used

NO.	ATTACK TYPE	TOOL	COMMAND	DESCRIPTION
1.	TCP SYN Flood	hping3	sudo hping3 -S --flood -V -p 80 [target IP]	Sends a continuous stream of SYN packets to port 80 of the target without completing the handshake, simulating a DoS attack
2.	ICMP Flood	hping3	sudo hping3 --icmp --flood [target IP]	Sends a continuous stream of ICMP Echo Request packets to overwhelm the target
3.	Port Scan	nmap	sudo nmap -sS -T4 -p 1-1024 [target IP]	Performs an aggressive TCP SYN scan across the first 1024 ports of the target, simulating reconnaissance activity

Each attack is run for a fixed duration of five minutes to generate a sufficient volume of labelled attack flows for evaluation. Between each attack run, a ten-minute period of normal traffic is generated by browsing the web, transferring files, and streaming video on the test laptop to simulate realistic benign network behaviour. This alternating pattern of normal and attack traffic allows the evaluation to measure how the IDS responds to transitions between normal and attack conditions, which is where the adaptive threshold mechanism is most critically tested.

3.6.2 Detection Rate Experiment

The detection rate experiment quantifies the accuracy of the IDS in detecting attack flows as attacks. The number of flows that are correctly identified as attacks for each attack type is noted, as well as the total number of attack flows generated during the simulation window. The rate of detection is then calculated as: **Detection Rate = (True Positives / (TP + FN)) x 100%.** It is conducted using both fixed threshold and adaptive threshold to compare the detection rate of both methods. For obvious attacks, particularly high volume attacks like TCP SYN flood attacks and ICMP flood attacks, a high detection rate is expected in both configurations. The more meaningful comparison is likely to occur during detection of the port scan, with lower traffic volume making threshold sensitivity more important. The target detection rate achieved in this project is at least 95% for all three attack types, which aligns with the target detection rate reported by Wijethilaka et al. (2025) for Random Forest classifier and Kuthada et al. (2021) for SVM classifier on similar attack categories.

3.6.3 False Positive Rate Experiment (Fixed vs. Adaptive Threshold)

The false positive rate experiment is the most important evaluation of this project as it represents the main contribution of the adaptive threshold mechanism: to reduce false positives under dynamic traffic conditions. This experiment measures the number of times that the IDS falsely detects an attack when normal traffic is being observed by both the fixed and adaptive threshold configurations. The False Positive Rate (FPR) is calculated as: False Positive Rate = (False Positives / (False Positives + True Negatives)) x 100% The experiment is carried out in three different traffic scenarios to evaluate the adaptive threshold under various traffic conditions. The first case is a steady normal traffic: In this case, the test laptop produces a steady background traffic at a moderate level. The second scenario is heavy normal traffic, and the test laptop acts as an internet browser, downloads big files, and streams video, thus creating an unexpected surge of legitimate traffic. One of the major reasons for using this scenario is to create false positives in the fixed threshold system because the increase in flow score moves closer to a fixed detection boundary. In the third case it is a combination of normal traffic and attack periods, which will test the ability to adapt to different traffic conditions in terms of speed for each of the different threshold configurations. The expected outcome is that the false positive rate (FPR) for the adaptive threshold is much less than the fixed threshold threshold while the traffic is heavy in the normal traffic scenario, because as traffic increases, the EMA baseline increases and the threshold remains above the higher normal traffic scores. This would directly confirm the main tenet of the project that an adaptive threshold has fewer false-positive in a dynamic network than a fixed threshold, as argued by Yu et al. (2024).

3.6.4 CPU and Memory Utilization Experiment

The CPU and Memory Utilization experiment record how much of the computational resources the IDS consumes while running on the Raspberry Pi. This is needed to ensure that the system will not use up too many resources on the Pi such that it will impair the functioning of the Pi as

a network gateway. The usage of CPU and memory is also measured continuously throughout the evaluation via the psutil Python library, which is already included in the dashboard system status view of Flask. Readings are captured every five seconds (the same time interval as the flow window) across all of the evaluation scenarios, resulting in a time-series record of the resource consumption. The experiment performs a test under four conditions: Idle (IDS running, with no traffic); Normal Traffic; Attack Traffic; Heavy Normal Traffic. The results for the four conditions are compared to determine if any condition causes the Pi to reach its limits of resources. During extended classification tasks, Musthafa et al. (2025) reported a 111.8% CPU load in the Raspberry Pi 3B+ processor.Musthafa et al. (2025) observed that the CPU load reached as high as 111.8% when the processor was used for sustained classification tasks on a different type of processor, the Raspberry Pi 3B+ which is less powerful than the Raspberry Pi 4 used here. With ARM Cortex-A72 processor handling the load, the Pi 4 has far more headroom, and CPU usage is likely to stay below 80% in all scenarios.

3.6.5 System Latency Experiment

The system response latency experiment measures the time elapsed between the end of a 5-second flow window and the point at which the corresponding alert, if any, is written to the SQLite database. This end-to-end latency covers feature extraction, model inference, EMA threshold computation, alert decision, and database write.

Latency is recorded for every processed flow window throughout the evaluation by timestamping the start of feature extraction and the completion of the database write for each window. The average latency, minimum latency, and maximum latency are then computed across all windows for each traffic scenario.

The target latency for this project is below 100 milliseconds per flow window. Wijethilaka et al. (2025) reported an average response time of 50 milliseconds for a Random Forest IDS on a Raspberry Pi 4 processing live traffic, which provides a directly comparable benchmark. Since the EMA computation adds a small additional step after classification, a slightly higher average latency than 50 milliseconds is acceptable as long as it remains well below 100 milliseconds.

3.6.6 Performance Metrics and Evaluation Criteria

This subsection outlines the performance indicators to consider when assessing if the project has achieved its objectives. There is a one-to-one relationship between each metric and one of the four experiments described in Sections 3.6.2 through 3.6.5, and a pass/fail threshold for each metric for setting a clear basis for a project's success or failure. All primary metrics must pass their own thresholds for success in a project. The choice and the definition of these metrics is based on the systematic review of the metrics for IDS performance carried out by Kadam and Verma (2025).

Metric Definitions and Formulae

The four main metrics are based on the confusion matrix, which keeps a record of all the outcomes of all the flow windows classified by the system. A True Positive (TP) is an attack flow which is correctly classified as an attack. A True Negative (TN) is a normal flow that is correctly identified as normal. A False Positive (FP) is a normal flow that is marked as an attack.

Attack flow that the system does not detect is False Negative (FN). Based on these four counts, all four metrics below are calculated.These four counts are used to calculate all four metrics below (Kadam and Verma, 2025; Huynh and Nguyen, 2024).

Detection Rate (DR) is the percentage of the real attack flows that the system is able to correctly detect. It is the same as Recall or True Positive Rate and equals: DR = TP / (TP+FN) x 100%. A high DR is crucial for the system to achieve its main objective of detecting real attacks. This project has a pass threshold of DR ≥ 95% for all three attacks (TCP SYN Flood, ICMP Flood, Port Scan), which aligns with the reported pass threshold in Wijethilaka et al. (2025) and Sow and Adda (2025).

False Positive Rate (FPR) is the ratio of normal flows to attack flows that are misclassified as attacks. It is calculated as: FPR = FP / (FP + TN) x 100%. This is the main index to validate the adaptive threshold mechanism. The pass threshold is FPR (Adaptive) < FPR (Fixed) in the heavy normal traffic scenario which shows measurable improvements over the fixed threshold baseline. According to Kadam and Verma (2025), the FPR is the most operationally critical metric in home and small office deployments where there is no security team present as false alarms affect user trust in the system.

CPU and Memory Utilisation: Average and peak percentage of CPU and RAM used by the IDS over a longer period of time. These values are taken directly from psutil each 5 seconds, and averaged for each traffic scenario. The pass threshold is CPU peak < 80% for all test scenarios which allows the Raspberry Pi to have enough CPU space to be used as a network gateway with the IDS. Jamshidi et al. (2025) list two key system-level feasibility indicators for considering edge-deployed IDS models: CPU usage and memory usage and, consistent with this project, they measure these two indicators under similar conditions.

System Response Latency is end-to-end processing latency from the end of the 5-second flow window until the completion of the EMA update, alert decision, and storage of the corresponding database write, which includes feature extraction, model inference, EMA update, alert decision, and storage. The average, minimum and maximum latency is measured per scenario. The pass threshold is average latency < 100ms per flow window for all the scenarios using the reference of 50ms (reported by Wijethilaka et al., 2025). Similarly, Jamshidi et al. (2025) includes end-to-end detection latency as part of their definition of inference latency and conclude that a response time of under 1.5 seconds is an acceptable time for real-time deployments of edge IDS, which supports the sub-100 ms threshold used here.

Observed Performance and Accuracy Results

The observed values for each metric for all experiments are recorded in table 3.10 below. The table should be filled in during the evaluation stage after implementation of the system is complete. The pass/fail judgement column is completed by comparing each observed value with the pass threshold listed above.

Table 3.10: Observed Performance and Accuracy Results

NO.	METRIC	PASS THRESHOLD	OBSERVED VALUE	EXPERIMENT (REF.)	PASS / FAIL
1.	Detection Rate — TCP SYN Flood	≥ 95%	TBD	§ 3.6.2	TBD
2.	Detection Rate — ICMP Flood	≥ 95%	TBD	§ 3.6.2	TBD

3.	Detection Rate — Port Scan	≥ 95%	TBD	§ 3.6.2	TBD
4.	FPR — Adaptive Threshold (Steady Traffic)	Record value	TBD	§ 3.6.3	TBD
5.	FPR — Adaptive Threshold (Heavy Traffic)	< FPR (Fixed)	TBD	§ 3.6.3	TBD
6.	FPR — Fixed Threshold (Heavy Traffic)	Record value (baseline)	TBD	§ 3.6.3	—
7.	Peak CPU Utilisation (Attack Traffic)	< 80%	TBD	§ 3.6.4	TBD
8.	Avg. RAM Usage (Attack Traffic)	Record value	TBD	§ 3.6.4	TBD
9.	Avg. System Response Latency	< 100 ms	TBD	§ 3.6.5	TBD

If the Detection Rate for all three attack types (rows 1-3) and the FPR Adaptive < FPR Fixed under heavy traffic (row 5) are all satisfied, the project is considered to have achieved its primary objective. Row 7 (CPU utilisation) and row 9 (latency) are secondary criteria, a failure on either does not necessarily mean project failure, but rather it means that there is a concern regarding resource constraint. In rows 4, 6, and 8, the data are observational and do not have pass/fail criteria. These are used for context when considering the main data.

3.7 Summary

This chapter presented the methodology adopted for the design and development of the Adaptive Threshold Intrusion Detection System using Random Forest on Raspberry Pi. The Waterfall model was selected as the project management methodology due to the clearly defined requirements, fixed timeline, and sufficient upfront planning time available for this project, and the four phases of Planning, Design, Development, and Evaluation were used to structure the remainder of the chapter.

The Planning phase set the basic decisions for the system, such as using CIC-IDS2017 as a training set, Random Forest as a classification method, EMA as the basis of adaptive threshold mechanism and the Raspberry Pi 4 as a hardware platform for deployment. During the Design phase, these choices led to a specific architecture, including three-layer system design, the network testbed topology, the feature extraction pipeline, the two-tier EMA threshold and learning window, and live detection phases, and the optional feature of IP auto-block mitigation with its respective diagram.
The Development phase explained how each of the design elements is implemented, ranging from Raspberry Pi OS configuration and setting up a Python environment, to offline model training, and the deployment of the model, from live traffic capture to the adaptive threshold module, to the Flask dashboard and the IP auto-block mechanism. Evaluation and Testing phase described the experiments to be performed after development is finished: detection rate, comparison of the rate of false positives between fixed threshold and adaptive threshold, CPU and memory usage, and time needed for system to react to DoS, ICMP flood, and port scan attack, simulated with hping3 and nmap in a controlled testbed.
These phases were presented on a 28-week Gantt chart, and were documented throughout all four phases. Combined, this method offers a logical and well-supported approach from the project's research gap, the lack of adaptive threshold mechanisms in current lightweight IDS implementations on constrained hardware, to a working system that can be shown to improve the ability to detect attacks over the case of fixed threshold approaches.

REFERENCES

Abu Bakar, R., & Kijsirikul, B. (2023). Enhancing Network Visibility and Security with Advanced Port Scanning Techniques. Sensors, 23(17), 7541. https://www.mdpi.com/1424-8220/23/17/7541

Adrian, R., Mandasari, R. D., & Alam, S. (2025). Design and Implementation of a Machine Learning-Based Adaptive IDS on Raspberry Pi for Smart Home Network Security. Jurnal Sains Dan Teknologi Industri, 22(2), 334. https://doi.org/10.24014/sitekin.v22i2.33485

Albasheer, H., Md Siraj, M., Mubarakali, A., Elsier Tayfour, O., Salih, S., Hamdan, M., Khan, S., Zainal, A., & Kamarudeen, S. (2022). Cyber-Attack Prediction Based on Network Intrusion Detection Systems for Alert Correlation Techniques: A Survey. Sensors, 22(4), 1494. https://www.mdpi.com/1424-8220/22/4/1494

Ali, T. E., Chong, Y.-W., & Manickam, S. (2023). Machine Learning Techniques to Detect a DDoS Attack in SDN: A Systematic Review. Applied Sciences, 13(5), 3183. https://www.mdpi.com/2076-3417/13/5/3183

Bassil, Y. (2012). A simulation model for the waterfall software development life cycle. International Journal of Engineering & Technology, 2(5), 1–7. https://arxiv.org/abs/1205.6904

Canavese, D., Mannella, L., Regano, L., & Basile, C. (2024). Security at the Edge for Resource-Limited IoT Devices. Sensors, 24(2), 590. https://www.mdpi.com/1424-8220/24/2/590

Devan, P., & Khare, N. (2020). An efficient XGBoost-DNN-based classification model for network intrusion detection system. Neural Computing and Applications, 32, 12499-12514. https://doi.org/10.1007/s00521-020-04708-x

Disha, R. A., & Waheed, S. (2022). Performance analysis of machine learning models for intrusion detection system using Gini Impurity-based Weighted Random Forest (GIWRF) feature selection technique. Cybersecurity, 5, Article 1. https://doi.org/10.1186/s42400-021-00103-8

Halimaa, A., & Sundarakantham, K. (2019). Machine Learning Based Intrusion Detection System. In Proceedings of the 3rd International Conference on Computing and Network Communications (CoCoNet 2019). https://ieeexplore.ieee.org/document/8862784

Huynh, T. T., & Nguyen, H. T. (2024). Effective multi-stage training model for edge computing devices in intrusion detection. International Journal of Computer Networks and Communications, 16(1), 1–18. https://doi.org/10.5121/ijcnc.2024.16102

Jamshidi, S., Nikanjam, A., Shahabi, N., Nafi, K. W., Khomh, F., Keivanpour, S., Herrero, R., Abdul Wahab, O., & Bellaïche, M. (2025). Think fast: Real-time IoT intrusion reasoning using IDS and LLMs at the edge gateway. arXiv preprint arXiv:2511.18230. https://arxiv.org/abs/2511.18230

Kadam, V., & Verma, R. (2025). Evaluating effectiveness: A critical review of performance metrics in intrusion detection system. Journal of Engineering Science and Technology Review, 18(1), 199–209. https://www.jestr.org/downloads/Volume18Issue1/fulltext201812025.pdf

Le, T.-T.-H., Kim, H., Kang, H., & Kim, H. (2022). Classification and explanation for intrusion detection system based on ensemble trees and SHAP method. Sensors, 22(3), 1154. https://www.mdpi.com/1424-8220/22/3/1154

Liu, L., Wang, P., Lin, J., & Liu, L. (2021). Intrusion detection of imbalanced network traffic based on machine learning and deep learning. IEEE Access, 9, 7550-7563. https://doi.org/10.1109/ACCESS.2020.3048198

Lu, C., Cao, Y., & Wang, Z. (2024). Research on intrusion detection based on an enhanced random forest algorithm. Applied Sciences, 14(2), 714. https://doi.org/10.3390/app14020714

Musthafa, M. B., Huda, S., Nguyen, T. T., Kodera, Y., & Nogami, Y. (2025). Optimized Ensemble Deep Learning for Real-Time Intrusion Detection on Resource-Constrained Raspberry Pi Devices. IEEE Access, 13, 113544–113556. https://doi.org/10.1109/access.2025.3584373

Petersen, K., Wohlin, C., & Baca, D. (2009). The waterfall model in large-scale development. In Lecture Notes in Business Information Processing (Vol. 32, pp. 386–400). Springer. https://doi.org/10.1007/978-3-642-02152-7_29

Pinto, A., Herrera, L.-C., Donoso, Y., & Gutierrez, J. A. (2023). Survey on Intrusion Detection Systems Based on Machine Learning Techniques for the Protection of Critical Infrastructure. Sensors, 23(5), 2415. https://www.mdpi.com/1424-8220/23/5/2415

Sai, K., Gupta, B., Hsu, C.-H., & Peraković, D. (2021). Lightweight Intrusion Detection System In IoT Networks Using Raspberry pi 3b+. https://ceur-ws.org/Vol-3080/7.pdf

Samantaray, M., Ram Chandra Barik, & Anil Kumar Biswal. (2024). A comparative assessment of machine learning algorithms in the IoT-based network intrusion detection systems. Decision Analytics Journal, 11, 100478 https://www.sciencedirect.com/science/article/pii/S2772662224000821

Santhosh Kumar, S. V. N., Selvi, M., & Kannan, A. (2023). A Comprehensive Survey on Machine Learning-Based Intrusion Detection Systems for Secure Communication in Internet of Things. Computational Intelligence and Neuroscience, 2023, 1–25. https://onlinelibrary.wiley.com/doi/10.1155/2023/8981988

Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108-116. https://doi.org/10.5220/0006639801080116

Sow, T. H., & Adda, M. (2025). Enhancing IDS performance through a comparative analysis of Random Forest, XGBoost, and deep neural networks. Machine Learning with Applications, 22, 100738. https://www.sciencedirect.com/science/article/pii/S2666827025001215

Tariq, S., Baruwal Chhetri, M., Nepal, S., & Paris, C. (2025). Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities. ACM Computing Surveys, 57(9), Article 224. https://dl.acm.org/doi/10.1145/3723158

Tekin, N., Acar, A., Aris, A., Uluagac, A. S., & Gungor, V. C. (2023). Energy consumption of on-device machine learning models for IoT intrusion detection. Internet of Things, 21, 100670. https://doi.org/10.1016/j.iot.2022.100670

Thakkar, A., & Lohiya, R. (2020). A Review of the Advancement in Intrusion Detection Datasets. Procedia Computer Science, 167, 636–645. https://www.sciencedirect.com/science/article/pii/S1877050920307961?via%3Dihub

Wijethilaka, R. W. K. S., Yapa, K., & Siriwardena, D. (2025). A cost effective machine learning based network intrusion detection system using Raspberry Pi for real time analysis. PLOS One, 20(12), e0331123. https://doi.org/10.1371/journal.pone.0331123

Wu, H., Shao, Z., Yang, F., Cheng, G., Hu, X., Ren, J., & Wang, W. (2023). PD-CPS: A practical scheme for detecting covert port scans in high-speed networks. Computer Networks, 231, 109825. https://www.sciencedirect.com/science/article/pii/S1389128623002700?via%3Dihub

Yang, G., Tang, C., & Liu, X. (2022). DualAC2NN: Revisiting and Alleviating Alert Fatigue from the Detection Perspective. Symmetry, 14(10), 2138. https://www.mdpi.com/2073-8994/14/10/2138

Yu, H., Yang, W., Cui, B., Sui, R., & Wu, X. (2024). Renyi entropy-driven network traffic anomaly detection with dynamic threshold. Cybersecurity, 7(64). https://link.springer.com/article/10.1186/s42400-024-00249-1